УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У НОВОМ САДУ

Денеш Сентмартони

Евалуација перформанси микросервисне апликације у Azure Kubernetes Service окружењу на примеру *redirect* апликације

МАСТЕР РАД
- Мастер академске студије -

Нови Сад, 2026

УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, **РБР**:		
Идентификациони број, **ИБР**:		
Тип документације, **ТД**:		Монографска документација
Тип записа, **ТЗ**:		Текстуални штампани материјал
Врста рада, **ВР**:		Завршни (Мастер) рад
Аутор, **АУ**:		Денеш Сентмартони
Ментор, **МН**:		др Себастијан Стоја, доцент
Наслов рада, **НР**:		Евалуација перформанси микросервисне апликације у Azure Kubernetes Service окружењу на примеру redirect апликације
Језик публикације, **ЈП**:		Српски/ ћирилица
Језик извода, **ЈИ**:		Српски
Земља публиковања, **ЗП**:		Република Србија
Уже географско подручје, **УГП**:		Војводина
Година, **ГО**:		2026
Издавач, **ИЗ**:		Ауторски репринт
Место и адреса, **МА**:		Нови Сад, Трг Доситеја Обрадовића 6
Физички опис рада, **ФО**: (поглавља/страна/ цитата/табела/слика/графика/прилога)		7/53/24/27/1/9/0
Научна област, **НО**:		Електротехничко и рачунарско инжењерство
Научна дисциплина, **НД**:		Примењени софтверски инжењеринг
Предметна одредница/Кључне речи, **ПО**:		Евалуација перформанси, микросервис, Azure Kubernetes Service
УДК		
Чува се, **ЧУ**:		У библиотеци Факултета Техничких Наука
Важна напомена, **ВН**:		
Извод, **ИЗ**:		У овом раду је испитан утицај хоризонталног и вертикалног скалирања на перформансе микросервисне апликације на примеру redirect апликације у окружењу Azure Kubernetes Service. Апликација је изложена степенастом оптерећењу генерисаном алатом k6, а у контролисаним експериментима мењани су број реплика, величина чвора апликације и величина чвора базе података, уз мерење капацитета, времена одговора и искоришћеност ресурса.
Датум прихватања теме, **ДП**:		14.09.2026.
Датум одбране, **ДО**:		
Чланови комисије, **КО**:	Председник:	др Александар Селаков, ванредни професор
	Члан:	др Дарко Чапко, редовни професор
	Члан, ментор:	др Себастијан Стоја, доцент

Потпис ментора

UNIVERSITY OF NOVI SAD ● **FACULTY OF TECHNICAL SCIENCES**
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, **ANO**:			
Identification number, **INO**:			
Document type, **DT**:		Monographic publication	
Type of record, **TR**:		Textual printed material	
Contents code, **CC**:		Master Thesis	
Author, **AU**:		Deneš Sentmartoni	
Mentor, **MN**:		Sebastijan Stoja, PhD, Assistant professor	
Title, **TI**:		Performance evaluation of a microservice application in an Azure Kubernetes Service environment using a redirect application as an example	
Language of text, **LT**:		Serbian	
Language of abstract, **LA**:		Serbian/English	
Country of publication, **CP**:		Serbia	
Locality of publication, **LP**:		Vojvodina	
Publication year, **PY**:		2026	
Publisher, **PB**:		Author reprint	
Publication place, **PP**:		Faculty of Tehnical Sciences, D. Obradovića 6, 21000 Novi Sad	
Physical description, **PD**: (chapters/pages/ref./tables/pictures/graphs/appendixes)		7/53/24/27/1/9/0	
Scientific field, **SF**:		Electrical and computer engineering	
Scientific discipline, **SD**:		Applied Software Engineering	
Subject/Key words, **S/KW**:		Performance evaluation, microservice, Azure Kubernetes Service	
UC			
Holding data, **HD**:		Library of the Faculty of Tehnical Sciences	
Note, **N**:			
Abstract, **AB**:		In this paper the impact of horizontal and vertical scaling on the performance of a microservice application is examined, using a redirect application as an example in an Azure Kubernetes Service environment. The application was subjected to stepwise increasing workloads generated using the k6 tool. In controlled experiments, the number of replicas, the size of the application node, and the size of the database node were varied, while capacity, response time, and resource utilization were measured.	
Accepted by the Scientific Board on, **ASB**:		September 14th, 2026	
Defended on, **DE**:			
Defended Board, **DB**:	President:	PhD Aleksandar Selakov, Associate professor	
	Member:	PhD Darko Čapko, Full professor	Menthor's sign
	Member, Mentor:	PhD Sebastijan Stoja, Assistant professor	

	УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА** 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Примењено софтверско инжењерство		
Студент:	Денеш Сентмартони	Број индекса:	E54/2023
Степен и врста студија:	Мастер академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	др Себастијан Стоја, доцент		

НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА
ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:

- проблем – тема рада;
- начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;

НАСЛОВ ЗАВРШНОГ РАДА:

Евалуација перформанси микросервисне апликације у Azure Kubernetes Service окружењу на примеру redirect апликације.

ТЕКСТ ЗАДАТКА:

Проучити микросервисну архитектуру, стратегије хоризонталног и вертикалног скалирања и начине мерења перформанси веб система. Развити микросервисну redirect апликацију, са инструментацијом за мерење пропусности, кашњења и искоришћености ресурса, и распоредити је у окружењу Azure Kubernetes Service. Дефинисати поступак мерења, профил оптерећења и критеријум капацитета, па извести експерименте у којима се мењају број реплика, величина чвора апликације и величина чвора базе података. На основу резултата утврдити која стратегија скалирања обезбеђује већи капацитет при једнакој укупној количини процесорских ресурса и у којој мери разлику објашњава број конекција према дељеној бази података.

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора

УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

_____

Кандидат

_____

Садржај

Списак слика **1**

Списак табела **2**

Списак листинга **3**

Списак скраћеница **4**

1. Увод **5**
1.1. Предмет истраживања 5
1.2. Циљ рада 6
1.3. Истраживачка питања 6
1.4. Метод истраживања 6
1.5. Очекивани допринос 7
1.6. Структура рада 7

2. Теоријске основе **8**
2.1. Микросервисна архитектура 8
2.1.1. Од монолитне ка микросервисној архитектури 8
2.1.2. Особине и изазови микросервисне архитектуре 8
2.2. Скалирање софтверских система 9
2.2.1. Хоризонтално скалирање 9
2.2.2. Вертикално скалирање 9
2.3. База података као ограничење скалирања 10
2.3.1. Конекција ка бази података 10
2.3.2. Повећање број конекција при хоризонталном скалирању 10
2.4. Оркестрација контејнера 10
2.4.1. Основни појмови система Kubernetes 11
2.5. Мерење перформанси веб система 12
2.5.1. Пропусност, кашњење и искоришћеност ресурса 12
2.5.2. Отворени и затворен модел оптерећења 12
2.5.3. Појам капацитета 12

3. Опис коришћених технологија и алата **13**
3.1. Go 13
3.2. Gin 13
3.3. pgx 14
3.4. PostgreSQL 14
3.5. RabbitMQ 15
3.6. Docker 15
3.7. Kubernetes 15
3.8. Microsoft Azure 16
3.9. Azure Kubernetes Service 16
3.10. Prometheus 17
3.11. cAdvisor 17
3.12. k6 18

4. Redirect Апликација **19**
4.1. Намена и функционални захтеви 19
4.2. Архитектура система 19
4.3. Сервиси 21
4.3.1. Urlshortener сервис 21
4.3.2. Redirect сервис 21
4.3.3. Analytics сервис 22
4.4. Модел података 22
4.5. Комуникација између сервиса 23

4.6. Инструментација ... 23
4.7. Контејнеризација ... 24
4.8. Распоређивање на Azure Kubernetes Service ... 25
4.9. Групе чворова ... 27
5. Методологија мерења ... 28
5.1. Променљиве и контролисани фактори ... 28
5.2. Скуп података и његово генерисање ... 29
5.3. Модел оптерећења ... 30
5.4. Дефиниција капацитета и критеријум засићења ... 30
5.5. Мерење величина и начин прикупљања ... 31
5.6. Ток једног мерења ... 32
6. Експерименти ... 34
6.1. Експеримент 1 ... 34
6.2. Експеримент 2 ... 34
6.3. Експеримент 3 ... 35
6.4. Експеримент 4 ... 37
6.5. Експеримент 5 ... 39
6.6. Одступања од плана и неважећа мерења ... 41
6.7. Анализа резултата и поређење ... 42
6.7.1. Пример 1 - Која компонента система троши највише процесорског времена по обрађеном захтеву и да ли заузеће радне меморије расте са оптерећењем? ... 42
6.7.2. Пример 2 - Како се перформанса апликације мења при хоризонталном скалирању, односно при повећању броја реплика уз непромењену величину чвора? ... 43
6.7.3. Пример 3 - Како се перформанса апликације мења при вертикалном скалирању, односно при повећању величине чвора уз непромењен број реплика? ... 44
6.7.4. Пример 4 - Која од двеју стратегија скалирања обезбеђује већи капацитет при једнакој укупној количини процесорских ресурса и колика је разлика перформанси међу њима? 46
6.7.5. Пример 5 - У којој мери утврђену разлику објашњава број конекција које реплике одржавају према дељеној бази података ? ... 47
7. Закључак ... 50
Литература ... 51
Биографија ... 53

Списак слика

Слика 4.2.1. Архитектура redirect апликације 20
Слика 6.7.1.1. Утрошено процесорско време по обрађеном захтеву у микросекундама 42
Слика 6.7.2.1. Време одговора сервиса Redirect као 95. перцентил 43
Слика 6.7.2.2. Чекање на конекцију по обрађеном захтеву 44
Слика 6.7.3.1. Време одговора сервиса Redirect као 95. перцентил 45
Слика 6.7.3.2. Чекање на конекцију по обрађеном захтеву 45
Слика 6.7.4.1. Време одговора сервиса Redirect као 95. перцентил при једнаком броју језгара, у милисекундама, на логаритамској скали 46
Слика 6.7.4.2. Обрађена стопа сервиса Redirect при једнаком броју језгара, у захтевима у секунди; 47
Слика 6.7.5.1. Време одговора сервиса Redirect као 95. перцентил у петом експерименту, у милисекундама 48
Слика 6.7.5.2. Чекање на конекцију по обрађеном захтеву у петом експерименту, у милисекундама, на логаритамској скали 48

Списак табела

Табела 4.1.1. Функционални захтеви и сервиси који имплементирају захтеве 19
Табела 4.4.1. Структура табеле веза 22
Табела 4.4.2. Структура табеле посета 23
Табела 4.6.1. Метрике доступне у сервисима 24
Табела 4.9.1. Групе чворова 27
Табела 5.1.1. Независне промењљиве 28
Табела 5.1.2. Зависне промењљиве 28
Табела 5.1.3. Контролисани фактори 29
Табела 6.1.1. Утрошено процесорско време по компоненти, у милијезгрима (средња вредност и стандардна девијација) 34
Табела 6.1.2. Заузеће радне меморије по компоненти, у мебибајтима (средња вредност и стандардна девијација) 34
Табела 6.2.1. Утрошено процесорско време по компоненти, у милијезгрима (средња вредност и стандардна девијација) 35
Табела 6.2.2. Заузеће радне меморије по компоненти, у мебибајтима (средња вредност и стандардна девијација) 35
Табела 6.3.1. Конфигурације 3-ег експеримента 35
Табела 6.3.2. Време одговора и обрађена стопа сервиса Redirect (средња вредност и стандардна девијација) 36
Табела 6.3.3. Чекање на конекцију према бази података (средња вредност и стандардна девијација) 36
Табела 6.3.4. Утрошено процесорско време сервиса Redirect (средња вредност и стандардна девијација) 37
Табела 6.3.5. Заузеће радне меморије сервиса Redirect (средња вредност и стандардна девијација) 37
Табела 6.4.1. Конфигурације 4-ог експеримента 37
Табела 6.4.2. Време одговора и обрађена стопа сервиса Redirect (средња вредност и стандардна девијација) 38
Табела 6.4.3. Чекање на конекцију према бази података (средња вредност и стандардна девијација) 38
Табела 6.4.4. Утрошено процесорско време сервиса Redirect (средња вредност и стандардна девијација) 39
Табела 6.4.5. Заузеће радне меморије сервиса Redirect (средња вредност и стандардна девијација) 39
Табела 6.5.1. Конфигурације 5-ог експеримента 39
Табела 6.5.2. Време одговора, обрађена стопа и капацитет сервиса Redirect (средња вредност и стандардна девијација) 40
Табела 6.5.3. Чекање на конекцију и трајање упита ка бази података (средња вредност и стандардна девијација) 40
Табела 6.5.4. Утрошено процесорско време базе података urlshortener (средња вредност и стандардна девијација) 41
Табела 6.5.5. Заузеће радне меморије базе података urlshortener (средња вредност и стандардна девијација) 41

Списак листинга

Листинг 4.7.1. Опис Dockerfile-а 25
Листинг 5.2.1. Наредба за стварање скупа података 29
Листинг 5.4.1. Дефиниција хистограма времена извршавања захтева 31

Списак скраћеница

Скраћеница	Пун назив
AMQP	*Advanced Message Queuing Protocol*
API	*Application Programming Interface*
HTTP	*HyperText Transfer Protocol*
JSONB	*JSON Binary*
K8s	*Kubernetes*
REST	*Representational State Transfer*
SQL	*Structured Query Language*
TCP	*Transmission Control Protocol*
URL	*Uniform Resource Locator*
vCPU	*virtual Central Processing Unit*
CB	средња вредност

1. Увод

Развој савремених система све чешће се ослања на микросервисну архитектуру, приступ у коме је апликација израђена као скуп малих, независних сервиса који међусобно комуницирају преко јасно дефинисаних интерфејса. За разлику од монолитне архитектуре, где се цела апликација развија, испоручује и скалира као једна целина, микросервисна архитектура омогућава да сваки сервис има сопствени животни циклус и сопствене технологије. Монолитна апликација је тешка за одржавање, и грешка у систему проузрокује да цео систем падне и престане да ради. У микросервисној архитектури ако се у једном сервису деси грешка онда само тај сервис престаје да ради а сви остали сервиси настављају да раде. Код иницијалне поделе сервиса треба гледати да се домен апликације подели на мање независне домене. Сваки сервис има само једну улогу. Сваки сервис треба да има сопствену базу података о којој ће чувати све своје податке. Ако софтверски инжењери прошире постојећи микросервис на начин да има више функционалности за решавање различитих проблема а не само једну онда у том случају постојећи сервис треба да се разбије на мање сервисе.

Практична примена апликација са микросервисном архитектуром се готово увек ослања на платформе за оркестрацију контејнера, међу којима је Kubernetes постао најшире прихваћено решење. Kubernetes је отвореног кода, али постоји више решења која нуде додатна окружења и алате и тиме сам систем проширују. Azure Kubernetes Service једно је од таквих окружења, које коришћење система Kubernetes поједностављује и олакшава. Код њега контролна раван (енгл. control plane) у целости одржава пружалац услуге, па корисник не подиже ни сервер програмског интерфејса ни базу стања, већ управља само радним чворовима [1]. Чворови се организују у групе, од којих свака има сопствени тип виртуелне машине и сопствени број чворова, а услуга је повезана са осталим деловима платформе, спремиштем слика, управљањем идентитетом и системом за надзор [2].

Повећање капацитета система остварује се на два начина. Хоризонтално скалирање подразумева повећање броја инстанци сервиса, при чему појединачна инстанца остаје непромењена. Вертикално скалирање подразумева додељивање веће количине ресурса постојећој инстанци. У пракси се хоризонтално скалирање узима као подразумевани избор, будући да се сматра природним за сервисе без стања и да га платформе за оркестрацију непосредно подржавају. Отворено је питање да ли је тај избор и мерљиво бољи од вертикалног, при једнакој укупној количини ресурса.

1.1. Предмет истраживања

Микросервиси се скалирају и вертикално и хоризонтално. Током хоризонталног скалирања додају се нове инстанце сервиса притом свака инстанца према бази података одржава сопствени скуп конекција, чија величина не зависи од броја језгара додељених инстанци, него од самог постојања инстанце. Из тога следи да број инстанци није само мера расположиве процесорске снаге, него истовремено и параметар оптерећења дељене базе података. Други вид скалирања је вертикално скалиранје, где се број инстанци не мења него се повећава процесорска снага. На овај начин број конекција инстанци према бази података остаје иста али опет се оптерећује база података. Предмет истраживања у овом раду јесте однос између конфигурације извршног окружења микросервисне апликације, коју чине број реплика и величина чвора, и капацитета који та апликација постиже, при непромењеној укупној количини процесорских ресурса и непромењеном дељеном слоју података.

1.2. Циљ рада

Циљ рада јесте да се емпиријски евалуира утицај конфигурације извршног окружења микросервисне апликације, коју чине број реплика и величина чвора, на пропусност, кашњење и искоришћеност ресурса у окружењу Azure Kubernetes Service. У оквиру тог циља потребно је пратити и упоредити искоришћеност процесора и заузеће радне меморије по појединачној компоненти система, и на основу тих мерења идентификовати компоненту која у свакој од посматраних конфигурација представља ограничење капацитета. Потребно је затим утврдити у којој мери разлику између двеју стратегија скалирања објашњава број конекција које реплике одржавају према дељеној бази података. Коначно, циљ је да се на основу прикупљених мерења непосредно упореде хоризонтално и вертикално скалирање и утврди која од двеју стратегија обезбеђује већи капацитет при једнакој укупној количини процесорских ресурса.

1.3. Истраживачка питања

Из наведеног циља изведена су следећа истраживачка питања:

1. Која компонента система троши највише процесорског времена по обрађеном захтеву и да ли заузеће радне меморије расте са оптерећењем?

2. Како се перформанса апликације мења при хоризонталном скалирању, односно при повећању броја реплика уз непромењену величину чвора?

3. Како се перформанса апликације мења при вертикалном скалирању, односно при повећању величине чвора уз непромењен број реплика?

4. Која од двеју стратегија скалирања обезбеђује већи капацитет при једнакој укупној количини процесорских ресурса и колика је разлика перформанси међу њима?

5. У којој мери утврђену разлику објашњава број конекција које реплике одржавају према дељеној бази података?

1.4. Метод истраживања

За остваривање постављеног циља примењен је контролисани експеримент. У ту сврху развијена је микросервисна апликација за преусмеравање URL адреса, која у овом раду има улогу мерног инструмента, а не самосталног доприноса. Апликација је распоређена на управљани Kubernetes кластер у оквиру услуге Azure Kubernetes Service, при чему се свака реплика сервиса извршава на сопственом чвору, чиме потрошња ресурса постаје недвосмислено приписива појединачним сервисима. Над тако постављеним системом извршава се унапред дефинисан профил оптерећења кроз низ конфигурација које се међусобно разликују само по броју реплика и величини чвора. Оптерећење се генерише алатом k6. Стопа долазака request-ova повећава се степенасто, са задржавањем на сваком степену, тако да свако мерење одговара устаљеном стању при познатој стопи. Прва два експеримента изведена су у три понављања, чиме је утврђена поновљивост мерења. Експерименти од трећег до петог изведени су у три покретању по конфигурацији, а пре сваког мерења систем се доводи у идентично почетно стање. Мере се прикупљају из три извора. Сервиси излажу сопствене метрике које прикупља Prometheus, потрошња процесора и заузеће радне меморије по контејнеру добијају се из система cAdvisor који излаже kubelet, а кашњења измерена на страни клијента шаље сам генератор оптерећења , софтверски алат k6.

1.5. Очекивани допринос

Од рада се очекују два доприноса:

1. Поређење односа хоризонталног и вертикалног скалирања микросервиса, добијен мерењем у окружењу Azure Kubernetes Service

2. Ррепоруке за избор конфигурације извршног окружења при ограниченој количини рачунарских ресурса, изведене непосредно из добијених мерења

1.6. Структура рада

Рад је организован у седам поглавља. После уводног поглавља, друго поглавље уводи појмове неопходне за разумевање проблема: микросервисну архитектуру, хоризонтално и вертикално скалирање, базу података као заједнички ресурс, оркестрацију контејнера и мерење перформанси веб система. Треће поглавље описује технологије и алате употребљене при изради апликације и мерењу, у мери у којој су потребни за праћење каснијих поглавља. Четврто поглавље описује апликацију која служи као мерни инструмент: њену архитектуру, сервисе, модел података, комуникацију између сервиса, инструментацију и начин распоређивања на кластер. Пето поглавље дефинише поступак мерења, односно променљиве и контролисане факторе, скуп података, модел оптерећења, дефиницију капацитета и критеријум засићења, као и ток једног мерења. Шесто поглавље приказује резултате пет експеримената и одступања од плана мерења, а у одељку 6.7 резултате тумачи и пореди, одговарајући редом на постављена истраживачка питања. Седмо поглавље сумира рад, издваја главне налазе и наводи правце даљег истраживања.

2. Теоријске основе

2.1. Микросервисна архитектура

Микросервисна архитектура представља архитектонски стил у коме је апликација изграђена као скуп малих сервиса, при чему се сваки сервис извршава у сопственом процесу и са осталим сервисима комуницира преко лаких механизама, најчешће преко мрежних интерфејса [1]. Сваки сервис покрива једну заокружену пословну способност и може се развијати, испоручивати и скалирати независно од осталих. Овај стил није настао као потпуно нова замисао, него као резултат дужег развоја начина на који се софтвер структурира. За разлику од сервисно оријентисане архитектуре, која није прецизно одредила ни величину сервиса ни начин њиховог раздвајања, микросервисна архитектура настала је из инжењерске праксе и уз њу постоје конкретни обрасци разлагања система [2].

2.1.1. Од монолитне ка микросервисној архитектури

Монолитна апликација развија се, преводи и испоручује као једна целина. Сав програмски код налази се у једном пројекту, а подаци по правилу у једној бази података. Такав приступ има стварне предности у раним фазама развоја: окружење за развој је једноставно, позиви између компоненти су локални позиви функција па немају мрежно кашњење, а измене над више табела обављају се у оквиру једне трансакције базе података [3]. Такође знање и искуство потребно да се изгради монолитна апликцаија је знатно мања. Наравно самим тим да апликација је један изворни код и не одвајају се сервиси и базе података и нема комуникације преко мреже између компоненти, узрокује да се знатно мање времена треба да се изгради и испоручи са истим бројем софтверски инжењера. У монолитној апликацији смо принуђени да се ослонимо само на један програмски језик што може већ на почетку развоја апликације да умањи перформансе и сам квалитет софтверског производа.

2.1.2. Особине и изазови микросервисне архитектуре

Најважнија особина микросервиса јесте могућност независне испоруке. Сервис се може изменити и поставити у извршно окружење без поновне испоруке осталих сервиса, под условом да се не мења уговор о комуникацији који су остали сервиси прихватили [3]. Сваки сервис требало би да поседује сопствену базу података коме други сервиси не приступају директно. Сваки сервис треба да управља само са својим подацима, ако треба да приступи бази података другог сервиса онда мора да контактира други сервис преко неког комуникационог протокола. Пошто се сервиси извршавају у одвојеним процесима, међусобно комуницирају преко мреже. Комуникација може бити синхрона, када позивалац чека одговор, најчешће преко протокола HTTP, или асинхрона, када се порука предаје посреднику и позивалац наставља рад не чекајући обраду [3]. Асинхрони начин користи се за операције које нису на критичној путањи одговора, чиме се време одзива штити од споријих делова система. Из независне испоруке следи и независно скалирање. Сваки сервис може добити онолико инстанци колико захтева његово оптерећење, независно од осталих [3]. Управо та особина предмет је испитивања у овом раду, будући да се независност скалирања у пракси показује као ограничена када више инстанци дели заједничке ресурсе. Микросервисна архитектура доноси и низ изазова који у монолитној апликацији не постоје. Позив који је раније био локални позив функције постаје позив преко мреже, па мора рачунати на кашњење, на делимичне отказе и на потребу за временским ограничењима и поновним покушајима [3]. Одржавање података је много тежа сваки сервис засебно ради са својим подацима са својом базом података. Не могу се сви уписи обавити у једној транзакцији, неко свака база им засебне транзакције па се мора водити рачуна и о конзистентности свих података у свим базама података. Праћење рад система постаје сложенији, будући да обрада једног корисничког захтева пролази кроз више сервиса, што захтева систематично прикупљање

метрика, дневника и трагова извршавања. Већи број јединица испоруке повећава оперативни трошак и практично намеће употребу платформе за оркестрацију. Захтев да сваки сервис има сопствену базу података често се ублажава, па више сервиса дели заједничко складиште, било због сложености раздвајања података, било због тога што су сервиси изведени из заједничког пословног домена [4].

2.2. Скалирање софтверских система

Под капацитетом система у овом раду подразумева се највећа стопа долазака захтева коју систем може да опслужи, а да при том задовољи унапред постављене захтеве у погледу времена одзива и удела неуспешних захтева. Када стварно оптерећење премаши капацитет, захтеви се задржавају у редовима чекања, време одзива нагло расте, а део захтева остаје неопслужен. Повећање капацитета постиже се додељивањем већег обима рачунарских ресурса. Разликују се две стратегије. Хоризонтално скалирање подразумева повећање броја инстанци које опслужују исто оптерећење, док вертикално скалирање подразумева додељивање већег обима ресурса постојећој инстанци [2]. Обе повећавају укупну количину расположивих ресурса, али то чине на начине који се битно разликују по претпоставкама које постављају пред апликацију и по ограничењима на која наилазе.

2.2.1. Хоризонтално скалирање

Хоризонтално скалирање остварује се повећањем инстанци сервиса, при чему се долазни захтеви расподељују међу њима посредством расподељивача оптерећења. Да би такав приступ уопште био могућ, инстанце морају бити међусобно заменљиве, односно сервис не сме да чува стање везано за појединачног корисника или појединачну сесију у сопственој меморији [1]. Стање се стога измешта у заједничко складиште, најчешће у базу података или у систем за привремено чување, коме све инстанце приступају равноправно. Предности овог приступа су вишеструке. Капацитет се повећава у корацима који нису ограничени величином појединачне машине, па се систем може ширити и преко граница најјаче доступне инстанце. Умножавање инстанци истовремено повећава расположивост, будући да отказ једне инстанце не прекида рад система него само умањује његов капацитет. У *cloud* окружењу број инстанци може се мењати током рада, чиме додељени ресурси прате променљиво оптерећење .Ограничења произлазе управо из претпоставке о заменљивости инстанци. Делови система који чувају стање не могу се умножавати на исти начин, па остају заједнички за све инстанце. Свака додатна инстанца сервиса тада не доноси само обрадну снагу, него и још једног корисника заједничких ресурса, чиме се терет постепено премешта са сервиса на заједнички слој. Тај механизам разматра се у одељку 2.3.

2.2.2. Вертикално скалирање

Вертикално скалирање остварује се додељивањем већег обима ресурса постојећој инстанци, најчешће већег броја процесорских језгара и веће количине радне меморије. У *cloud* окружењу то практично значи прелазак на већи тип виртуелне машине, док у клустеру за оркестрацију контејнера значи да се под извршава на чвору веће величине. Главна предност јесте у томе што овај облик скалирања не поставља додатне захтеве пред апликацију. Сервис не мора бити без стања, не мора постојати расподељивач оптерећења, нити се уводе нови учесници у комуникацији са заједничким ресурсима. Број веза које апликација одржава према бази података остаје непромењен, будући да се број инстанци не мења. Сложеност система остаје иста, а мења се само подлога на којој се извршава. Ограничења су, међутим, стварна. Величина појединачне машине има горњу границу, па се капацитет не може повећавати неограничено. Промена величине инстанце по правилу захтева поновно покретање, што уводи прекид у раду. Апликација уз то мора бити у стању да искористи додата језгра, што претпоставља упоредно извршавање унутар самог процеса; уколико је обрада ограничена на једну нит, додатна језгра остају неискоришћена. Једна велика инстанца представља јединствену тачку отказа, чиме је расположивост мања него код скупа мањих инстанци.

2.3. База података као ограничење скалирања

У претходном одељку објашњени су типови скалиранја. Али и вертикално и хоризонтално скалирање зависе од базе података. У посматраном систему база података је релациона и, за разлику од сервиса, не умножава се са оптерећењем у виду броја реплика. Разлози за то нису техничке природе. Повећавање инстанци базе података захтевало би усаглашавање података међу инстанцама, што за операције уписа уводи потребу за договором о редоследу измена и трошак који расте са бројем примерака. Уобичајена решења, попут репликације у којој један примерак прима уписе а остали опслужују читања, захтевају да апликација разликује путању читања од путање уписа и да толерише заостајање реплика за изворним примерком. Због тога се, нарочито у системима мањег обима, задржава једна инстанца базе података коју дели више сервиса и више њихових реплика [5].

2.3.1. Конекција ка бази података

Приступ бази података одвија се преко конекција. У систему PostgreSQL свака конекција опслужује се засебним процесом на страни сервера, који се ствара при успостављању везе и укида при њеном раскидању. Успостављање конекције стога није јефтина операција, будући да обухвата успостављање мрежне везе, проверу идентитета, стварање процеса и почетно подешавање сесије. Када би се веза успостављала за сваки долазни захтев, тај трошак би у систему са кратким упитима надмашио трошак саме обраде. Из тог разлога апликације користе скуп унапред успостављених конекција. Скуп се образује при покретању инстанце и одржава током њеног рада, а појединачни захтев позајмљује конекцију, извршава упит и враћа је у скуп. Скуп се описује са неколико параметара, од којих су најважнији најмањи и највећи број конекција које одржава. Најмањи број одређује колико се веза држи отвореним и када оптерећења нема, чиме се избегава трошак поновног успостављања, док највећи број ограничава колико веза инстанца сме истовремено да заузме. Понашање скупа при засићењу непосредно утиче на време одзива. Када су све конекције заузете, наредни захтев не добија везу одмах, него чека да се нека ослободи. То чекање није део извршавања упита и не види се у мерењима на страни базе података, али се сабира са временом одзива које корисник осећа.

2.3.2. Повећање број конекција при хоризонталном скалирању

Скуп конекција припада појединачном процесу, а не систему у целини. Свака реплика сервиса одржава сопствени скуп, независан од скупова осталих реплика. Укупан број конекција према бази података стога представља производ броја реплика. Код хоризонталног скалирања повећавањем проје реплика повећава се и број кеонекција. Због тога се уз број заузетих конекција прати и укупно време проведено у чекању на конекцију, будући да тек оно показује да је скуп постао ограничење. За базу података нису урађене додатне оптимизације, тако да број конекција не узрокује линеарни раст броја извршавања захтева ка бази, зато што у једом тренутку само једна нит може да уписује па остали захтеви чекају. Док код вертикалног скалирања, повећање броја језгара додељених једној инстанци не мења број инстанци, па се ни број скупова конекција не мења. Разлика између двеју стратегија тиме није само у облику хардвера на коме се извршавају, него и у томе што само једна од њих мења оптерећење заједничког слоја.

2.4. Оркестрација контејнера

Микросервисна архитектура повећава број јединица које треба поставити, надгледати и одржавати. Уместо једне апликације, у извршном окружењу налази се више процеса, од којих сваки има сопствену верзију, сопствену конфигурацију и сопствене захтеве у погледу ресурса. Ручно управљање таквим системом брзо постаје неизводљиво, због чега се микросервисна архитектура у пракси готово увек ослања на аутоматизацију. Основу те аутоматизације чине контејнери. Контејнер обједињује програм и све његове зависности у једну слику која се

извршава изоловано, користећи механизме језгра оперативног система за раздвајање процеса. За разлику од виртуелне машине, контејнер не садржи сопствено језгро него дели језгро домаћина, због чега се покреће знатно брже и троши мање ресурса. Пошто слика садржи све зависности, исти контејнер понаша се истоветно на радној станици и у *cloud* окружењу, што је предуслов поновљивости мерења. Оркестратор контејнера преузима одговорност за распоређивање контејнера на расположиве машине, за одржавање задатог броја примерака, за замену отказалих примерака, за откривање сервиса и за постепену испоруку нових верзија, као што је Azure Kubernetes Service који ће се користити у овом раду.

2.4.1. Основни појмови система Kubernetes

Kubernetes је оркестратор контејнера који је постао најшире прихваћено решење у овој области. Заснива се на декларативном моделу управљања, у коме корисник описује жељено стање система, а не поступак којим се то стање постиже. Управљачки слој непрекидно упоређује жељено стање са стварним и предузима радње које их изједначавају. Управљати њиме можемо преко терминала или преко веб странице. Уколико, на пример, откаже једна од четири тражене инстанце, систем сам покреће нову, без спољашње интервенције. Кластер чине управљачки слој и радни чворови. Управљачки слој чува жељено стање и доноси одлуке о распоређивању, док се на радним чворовима заиста извршавају контејнери. Логичко раздвајање ресурса унутар кластера остварује се именским просторима, чиме се објекти једне апликације одвајају од објеката других апликација на истом кластеру. Најмања јединица која се распоређује назива се под. Под обухвата један или више контејнера који деле мрежни именски простор и простор за складиштење, због чега се контејнери унутар истог пода могу обраћати један другом преко локалног домаћина. У већини случајева под садржи тачно један контејнер, док се више њих користи када помоћни процес мора да дели окружење са главним. Подови се ретко описују непосредно. Уместо тога користи се објекат за испоруку, који наводи жељени број примерака и предложак пода према коме се они стварају. Тај објекат посредно управља скупом реплика, који одржава задати број подова и обезбеђује поступну замену примерака при испоруци нове верзије. Пошто подови настају и нестају, а са њима се мењају и њихове мрежне адресе, приступ им се остварује преко објекта сервиса, који пружа постојану адресу и расподељује долазни саобраћај међу подовима који му припадају. Сваком контејнеру могу се придружити захтевана и највећа количина ресурса. Захтевана количина учествује у одлуци о распоређивању, будући да распоређивач бира чвор на коме је таква количина слободна, док највећа количина ограничава потрошњу током рада. Одлуку о томе на који ће чвор под бити постављен доноси распоређивач. Поступак се одвија у две фазе: најпре се одбацују чворови који не задовољавају постављене услове, а затим се преостали рангирају и бира се најбоље оцењени. Уколико ниједан чвор не задовољава услове, под остаје у стању чекања све док се услови не испуне. Најједноставније ограничење остварује се избором чвора према ознакама. Чворовима се додељују ознаке у облику пара кључа и вредности, а поду се наводи који пар мора бити задовољен, чиме се под усмерава на одређену групу чворова. Тим механизмом раздвајају се улоге унутар кластера, тако да, на пример, подови апликације не деле чворове са подовима базе података. Сложенија ограничења изражавају се афинитетом и одбојношћу међу подовима. Афинитет налаже да се под постави на чвор на коме се већ налази под са одређеном ознаком, док одбојност налаже супротно. Оба се могу задати као обавезна, у ком случају под остаје нераспоређен уколико услов не може бити испуњен, или као пожељна, у ком случају услов утиче на рангирање али није неопходан. Опсег у коме се услов проверава одређује се кључем топологије; када тај кључ одговара имену чвора, услов се примењује на нивоу појединачне машине. Комбинацијом обавезне одбојности према подовима исте ознаке и кључа топологије који одговара имену чвора постиже се да се на једном чвору налази највише један под из посматраног скупа. Такво распоређивање у мерењима перформанси има посебан значај, јер потрошња ресурса измерена на чвору тада припада тачно једној компоненти система, без удела суседних процеса. Цена таквог решења јесте то што број чворова мора бити најмање једнак броју подова у истом скупу, због чега свака промена броја реплика повлачи и одговарајућу промену броја чворова.

2.5. Мерење перформанси веб система

Поређење различитих конфигурација има смисла само уколико су унапред одређене величине које се мере и услови под којима се мерење обавља. У наставку су изложени појмови на којима то поређење почива, док је конкретан поступак мерења, са дефиницијама појединачних величина и вредностима прагова, дат у поглављу 5.

2.5.1. Пропусност, кашњење и искоришћеност ресурса

Пропусност представља број захтева које систем успешно опслужи у јединици времена. Она описује количину обављеног посла, али сама по себи не говори ништа о томе колико је корисник чекао на одговор.

Кашњење представља време протекло од упућивања захтева до пријема одговора. За разлику од пропусности, кашњење није једна вредност него расподела, будући да се појединачни захтеви разликују по трајању. Приказ помоћу средње вредности прикрива понашање у горњем делу расподеле, где се и налазе захтеви који корисничко искуство чине лошим, због чега се уместо просека наводе перцентили, најчешће деведесет пети и деведесет девети.

Искоришћеност ресурса означава удео капацитета неког ресурса који је у датом тренутку заузет, при чему се најчешће прате процесорско време и радна меморија. Однос између искоришћености и кашњења није линеаран: како се искоришћеност приближава јединици, време чекања у реду расте нагло, па и мало повећање оптерећења изазива велики пораст кашњења.

2.5.2. Отворени и затворен модел оптерећења

Оптерећење се при мерењу може задавати на два начина, који се битно разликују по томе шта одређује тренутак упућивања наредног захтева. У затвореном моделу задат је број истовремених корисника, при чему сваки од њих упућује наредни захтев тек пошто прими одговор на претходни. Стопа долазака тако зависи од времена одзива система: што систем спорије одговара, то мање захтева стиже. У отвореном моделу захтеви пристижу према унапред задатој стопи, независно од тога колико систему треба да одговори. Разлика има непосредне последице по резултат мерења. Затворени модел смањује оптерећење управо у тренутку када систем почиње да заостаје, чиме прикрива погоршање и систематски потцењује кашњења у горњем делу расподеле.

2.5.3. Појам капацитета

При постепеном повећању оптерећења пропусност најпре расте приближно сразмерно понуђеној стопи, док расположиви ресурси задовољавају потражњу. У једном тренутку неки од ресурса достиже засићење, након чега пропусност престаје да расте, кашњење нагло расте, а при даљем повећању оптерећења појављују се и неуспешни захтеви. Због тога се капацитет не одређује као највећа остварена пропусност. Пропусност постигнута уз време одзива које корисник не прихвата не представља употребљив капацитет. Уместо тога, капацитет се одређује као највећа стопа долазака при којој систем и даље задовољава унапред постављени услов квалитета, изражен горњом границом кашњења и највећим допуштеним уделом неуспешних захтева. Тако одређен капацитет представља једну бројчану вредност по конфигурацији, што омогућава непосредно поређење различитих конфигурација.

3. Опис коришћених технологија и алата

У овом поглављу описане су технологије и алати употребљени за израду и мерење система који је предмет рада.

3.1. Go

Go је компајлирани програмски језик опште намене, настао у компанији Google и јавно објављен 2009. године; аутори језика су Robert Griesemer, Rob Pike и Ken Thompson [6]. Језик је осмишљен као одговор на сложеност великих програмских система, па је синтакса намерно сведена: нема наслеђивања, нема преклапања оператора и постоји само један облик петље. Провера типова је статичка и обавља се при превођењу, а меморијом управља сакупљач отпадака, чиме се уклања читава класа грешака при раду са показивачима.

Превођењем настаје самостални извршни програм који у окружењу извршавања не захтева виртуелну машину нити додатне библиотеке. Превођење за друге оперативне системе и процесорске архитектуре изводи се са истог рачунара, без посебног ланца алата. Уз језик долази и јединствен скуп алата за превођење, испитивање, обликовање изворног кода и управљање зависностима, чиме се уклања потреба за спољним системима за изградњу.

Најизразитија особина језика јесте модел конкурентности. Основна јединица конкурентног извршавања назива се goroutine и представља нит којом управља извршно окружење језика, а не оперативни систем. Почетни стек goroutine износи свега неколико килобајта и расте по потреби, док извршно окружење више хиљада горутина мултиплексира преко малог броја нити оперативног система [7]. Горутине међусобно размењују податке каналима, према моделу комуникационих секвенцијалних процеса, чиме се заједничко стање и потреба за закључавањем своде на најмању меру. Сакупљач отпадака ради упоредо са програмом и пројектован је тако да паузе буду кратке, што је значајно за програме који одговарају на велики број истовремених захтева.

3.2. Gin

Gin је framework за израду HTTP сервиса у програмском језику Go. Намењен је изради сервиса заснованих на стилу REST, веб апликација и микросервиса, а нарочито случајевима у којима је брзина одговора битна [8].

Проналажење руте засновано је на структури радикс стабла. Руте се складиште као стабло заједничких префикса, па се време проналажења не повећава линеарно са бројем регистрованих рута. Сам поступак поклапања руте изведен је тако да у уобичајеном случају не захтева алокацију на heap-у, чиме се смањује притисак на сакупљач отпадака при великом броју захтева у секунди [8].

Обрада захтева организована је као ланац middleware-ова. Пошто се одговор кроз исти низ враћа назад, сваки међуслој може да одради део посла и пре и после саме обраде. Тако се послови који су исти за све руте као што су бележење догађаја, хватање неочекиваних грешака, провера идентитета или мерење трајања обраде пишу само на једном месту, уместо да се понављају у свакој функцији за обраду. Додатно нуди груписање рута по заједничком префиксу, везивање улазних података из тела захтева, упитног дела адресе или путање у структуре језика Go, проверу исправности тих података помоћу ознака над пољима структуре, као и обједињено пријављивање грешака.

3.3. pgx

pgx је управљачки програм и скуп алата за приступ систему PostgreSQL из програмског језика Go. За разлику од управљачких програма који раде преко стандардног интерфејса, pgx самостално имплементира мрежни протокол система PostgreSQL [9]. Тиме добија приступ особинама које кроз општи интерфејс нису доступне: бинарном формату преноса података, који уклања претварање вредности у текстуални облик и назад, сопственом кеширању припремљених упита и непосредном пресликавању типова система PostgreSQL у типове језика Go, укључујући низове, распоне и тип JSONB. Уз то, библиотека нуди и слој усклађен са интерфејсом базе података, за случајеве у којима је преносивост важнија од наведених могућности.

Саставни део библиотеке је и скуп конекција. Успостављање конекције ка бази података је скупа радња, јер обухвата успостављање мрежне везе, проверу идентитета и припрему стања на серверу, па се конекције не отварају по захтеву већ се држе отворене и додељују редом. Величина скупа задаје се доњом и горњом границом; доња граница одређује колико конекција се одржава отвореним и када нема посла, а горња колико их највише сме постојати истовремено.

Када се конекција тражи, а ниједна слободна не постоји, захтев се блокира док се нека конекција не ослободи или док се, ако горња граница још није достигнута, нова не успостави. Скуп о свом раду води бројно стање и излаже га позивачу: укупан број отворених конекција, број тренутно заузетих, подешену горњу границу, број заузимања која су морала да чекају и укупно време проведено у чекању [9]. Те величине омогућавају да се надметање за конекције посматра бројчано, а не само посредно кроз повећано кашњење.

3.4. PostgreSQL

PostgreSQL је објектно-релациони систем за управљање базом података. Настао је из истраживачког система POSTGRES, развијеног осамдесетих година на Универзитету Калифорније у Берклију, чији је циљ био да релациони модел прошири корисничким типовима података, правилима и наслеђивањем над табелама [10]. Систем обезбеђује атомичност, конзистентност, изолацију и трајност трансакција, прати SQL стандард у високој мери и подржава полуструктуриране податке у бинарном типу JSONB, над којим је могуће правити индексе.

Истовремени приступ решава се вишеверзијским управљањем конкурентношћу. Измена реда не мења постојећу верзију него ствара нову, док стара остаје видљива трансакцијама започетим раније. Последица је да читање не блокира писање нити писање блокира читање, па се закључавање своди на случајеве стварног сукоба измена. Трајност се обезбеђује дневником уписа унапред: промена се најпре уписује у дневник, а тек потом у саме датотеке података, чиме је после прекида рада могућ опоравак у конзистентно стање. Поред подразумеваног B-стабла, систем нуди и индексне структуре прилагођене текстуалној претрази, просторним подацима и типу JSONB, а функционалност се проширује додацима који се уграђују без измене језгра система.

Архитектуру сервера карактерише то што се за сваку клијентску конекцију покреће засебан процес оперативног система. Тај процес има сопствену меморију за сортирање, разврставање и међурезултате, независну од заједничког кеша страница. Укупан број таквих процеса ограничен је параметром max_connections, а свака отворена конекција троши ресурсе и када се преко ње не извршава ниједан упит [11]. Из тога следи да конекција ка бази података није бесплатан ресурс који се може произвољно умножавати, због чега се у системима са великим бројем клијената између апликације и базе по правилу поставља скуп конекција.

3.5. RabbitMQ

RabbitMQ је посредник у размени порука (енг. message broker) отвореног кода. Написан је у програмском језику Erlang, чије је извршно окружење пројектовано за системе са великим бројем лаких процеса, изолацијом грешака и високим захтевима за расположивошћу. Основни протокол јесте AMQP 0-9-1, бинарни протокол над TCP везом у коме се преко једне мрежне везе мултиплексира више логичких канала, тако да више токова размене дели исту везу [12].

Модел размене чине exchange, queue и веза међу њима. Произвођач поруку никада не шаље реду непосредно, већ exchange-у, који је на основу кључа усмеравања и врсте размењивача прослеђује једном реду, већем броју редова или ниједном [13]. Постоји више врста размењивача: direct, који поруку усмерава по тачном поклапању кључа, topic, који поруку прослеђује свим повезаним редовима, fanout, који кључ пореди по обрасцу, и headers, који одлуку доноси на основу заглавља поруке. Потрошач затим поруке преузима из реда.

Таквим посредовањем произвођач и потрошач се раздвајају: ниједан не мора знати за постојање оног другог, нити морају истовремено бити доступни, јер порука у реду чека док потрошач не буде спреман да је преузме. Поузданост се обезбеђује потврђивањем пријема, при чему се порука уклања из реда тек пошто је потрошач потврди, и трајношћу редова и порука, чиме садржај преживљава поновно подизање посредника. Више примерака посредника може радити у кластеру, а уз систем долази и додатак са управљачким приступом преко прегледача, који приказује стање редова, проток порука и број повезаних потрошача.

3.6. Docker

Docker је скуп алата за паковање и извршавање апликација у контејнерима. Контејнер представља изоловано окружење за извршавање процеса, изведено механизмима језгра оперативног система Linux. Именски простори процесу ограничавају видљивост система датотека, мреже, процеса и корисника, тако да процес види само себи додељени део система, док контролне групе ограничавају колико процесорског времена, меморије и улазно-излазног пропусног опсега процес сме да потроши [14].

За разлику од виртуелних машина, које подижу целовит гостујући оперативни систем над хипервизором, контејнери деле језгро домаћина. Отуда немају ни време подизања ни меморијски трошак гостујућег система, а мерења показују да је потрошња ресурса контејнера у већини случајева блиска извршавању непосредно на домаћину [15]. Цена те лакоће јесте слабија изолација, јер пропуст у заједничком језгру погађа све контејнере на истом домаћину.

Слика контејнера описује се датотеком у којој је сваки корак изградње наведен посебном инструкцијом, а сваки корак ствара нови слој система датотека. Слојеви су непроменљиви и деле се међу сликама, па се при испоруци преноси само оно што је стварно измењено. Слике се чувају у спремишту одакле се преузимају на чворове који их извршавају. Вишестепена изградња омогућава да се превођење обави у једној слици, која садржи преводилац и развојне алате, а да се у коначну слику пренесе само добијени извршни програм, чиме коначна слика остаје мала. Формат слика и начин њиховог извршавања стандардизовани су кроз иницијативу Open Container Initiative, па нису везани за једну имплементацију. Сам Docker написан је у програмском језику Go.

3.7. Kubernetes

Kubernetes, познат и под скраћеницом K8s, јесте систем отвореног кода за аутоматско распоређивање, скалирање и надзор контејнеризованих апликација. Настао је у компанији Google на основу вишегодишњег искуства са интерним системима Borg и Omega, одакле су

преузети и разрађени појмови попут декларативног описа жељеног стања, груписања сродних контејнера у једну јединицу распоређивања и раздвајања корисника од конкретног чвора на ком се посао извршава [16].

Систем се дели на контролну раван и радне чворове. Контролну раван чине сервер програмског интерфејса, кроз који пролази свака измена стања, расподељивач, који новим јединицама рада бира чвор, управљачки процеси, који стварно стање непрекидно приближавају жељеном, и база стања etcd, у којој се цела конфигурација кластера чува. На сваком радном чвору налазе се агент kubelet, који покреће и надгледа контејнере, мрежни посредник и извршно окружење контејнера [17].

Рад је организован око објеката које корисник описује декларативно, наводећи жељено стање, а не поступак којим се оно постиже. Под (енг. pod) је најмања јединица распоређивања и садржи један или више контејнера који деле мрежни простор и складиште. Скуп примерака (енг. ReplicaSet) одржава задати број истоветних подова и поново их подиже када неки престане са радом, а објекат распоређивања (енг. Deployment) над њим управља постепеном сменом верзија и повратком на претходну. Сервис групи подова додељује постојану адресу и оптерећење распоређује међу њима, чиме позивалац не мора знати где се поједини под налази. Подешавања и тајне вредности издвајају се у засебне објекте, а именски простори кластер деле на логичке целине.

Сваком контејнеру задају се тражени и највећи допуштени износи процесорског времена и меморије. Тражени износ расподељивач користи при избору чвора, а највећи допуштени спроводи се контролним групама током извршавања. Повезивање објеката изводи се ознакама и изборницима, чиме се остварује и усмеравање подова на одређене чворове.

3.8. Microsoft Azure

Microsoft Azure је платформа за рачунарство у *cloud* окружењу коју развија компанија Microsoft. Према општеприхваћеној дефиницији, рачунарство у *cloud* окружењу подразумева модел приступа заједничком скупу подесивих рачунарских ресурса који се могу брзо доделити и ослободити уз најмању меру интеракције са пружаоцем услуге. Дефиниција наводи пет суштинских особина, међу којима су самоуслуживање по потреби, еластичност и мерена потрошња, и три модела услуге: инфраструктуру, платформу и софтвер као услугу [18].

Ресурси платформе распоређени су по регионима, односно географски раздвојеним групама дата центара, а унутар региона по зонама расположивости, које се напајају и хладе независно једна од друге. Обрачун се врши према стварном трајању и обиму коришћења ресурса.

Виртуелне машине сврстане су у породице према намени, при чему се унутар исте породице мења само величина, а генерација процесора и однос између броја језгара и меморије остају исти. Породица Dsv5 припада машинама опште намене и нуди уравнотежен однос процесорске снаге, меморије и привременог складишта; величине унутар породице разликују се по броју виртуелних језгара и припадајућој меморији [19].

3.9. Azure Kubernetes Service

Azure Kubernetes Service јесте управљана услуга система Kubernetes на платформи Azure. Control plane у целости одржава пружалац услуге, који се стара о њеној расположивости, надоградњи и безбедносним исправкама, док корисник управља само радним чворовима. У бесплатном нивоу услуге control plane се не наплаћује, па се плаћају само виртуелне машине

радних чворова; плаћени ниво доноси уговорену расположивост контролне равни и подршку већим кластерима [20].

Чворови се организују у групе чворова (енг. node pool). Свака група има сопствени тип виртуелне машине, сопствени број чворова и сопствене ознаке, што омогућава да у истом кластеру постоје чворови различитих величина и намена, а да се одређени послови усмеравају само на одређену групу. Број чворова у групи мења се ручно или самоподешавајућим механизмом, који чворове додаје када подови не могу да се распореде, а уклања их када дуже време остану неискоришћени.

Услуга је повезана са осталим деловима платформе: са спремиштем слика контејнера, са системом за управљање идентитетом, са виртуелним мрежама и са системом за надзор. Надоградња верзије система Kubernetes изводи се постепеном сменом чворова, а могуће ју је препустити и аутоматским каналима, који кластер сами одржавају на новијим издањима.

3.10. Prometheus

Prometheus је систем отвореног кода за прикупљање и чување временских низова, намењен надзору система и апликација. Прикупљање се заснива на моделу повлачења: сервер у задатим размацима сам приступа унапред одређеној путањи сваког надгледаног примерка и преузима затечене вредности, уместо да чека да му их примерак пошаље [21]. Надгледани примерци проналазе се механизмом откривања, који се повезује са оркестратором или сервисом за разрешавање имена, па списак циљева прати стварно стање система и не мора се одржавати ручно.

Подаци се чувају у уграђеној бази временских низова. Низ је једнозначно одређен називом метрике и скупом ознака, при чему свака различита комбинација вредности ознака ствара посебан низ. Над низовима се поставља упит језиком PromQL, који подржава изборе по ознакама, рачунање брзине пораста, сабирање по ознакама и рачунске радње међу низовима.

Подржане су четири врсте метрика. Бројач је вредност која може само да расте и по правилу се не чита непосредно, већ се из њега рачуна пораст у јединици времена. Мерач је вредност која слободно расте и опада. Хистограм расподелу измерених вредности бележи у унапред задатим опсезима, при чему сваки опсег носи број вредности мањих или једнаких својој горњој граници; из таквог хистограма функцијом histogram_quantile процењује се квантил, уз линеарну интерполацију унутар опсега, због чега тачност процене зависи од избора граница [22]. Сажетак квантиле рачуна на страни примерка, чиме губи могућност сабирања по више примерака. Поред прикупљања повлачењем, сервер прихвата и удаљени упис, којим спољни алати своје метрике предају на чување, а уз систем долази и засебна компонента за обраду упозорења.

3.11. cAdvisor

cAdvisor (скраћено од Container Advisor) јесте компонента за прикупљање и анализу података о потрошњи ресурса контејнера. Развијена је у компанији Google и уграђена је у агента kubelet система Kubernetes, па на чворовима тог система не захтева посебну инсталацију [23].

Компонента самостално открива све контејнере који се извршавају на чвору и за сваки од њих прикупља податке о утрошеном процесорском времену, заузећу радне меморије, промету над системом датотека и мрежном промету. Подаци се читају из контролних група и осталих механизама језгра који већ воде евиденцију о потрошњи, па само прикупљање уноси мало додатног оптерећења. Прикупљене вредности излажу се на путањи коју преузима систем за надзор, у облику бројача и мерача. Учесталост очитавања подешава се сама, у зависности

од тога колико се стање контејнера мења, и креће се у распону од десет до петнаест секунди. На управљаним услугама тај распон по правилу није изложен кориснику на подешавање, чиме је временска резолуција података о потрошњи ресурса ограничена одозго.

3.12. k6

k6 је алат отвореног кода за испитивање под оптерећењем. Написан је у програмском језику Go и користи малу количину рачунарских ресурса за опонашање великог броја корисника, док се сценарији пишу у програмском језику JavaScript и извршавају уграђеним преводиоцем [24]. Тиме се једноставност писања сценарија у скриптном језику спаја са малом потрошњом ресурса самог генератора, што омогућава да један примерак алата ствара оптерећење које би иначе захтевало више машина.

Оптерећење се задаје извршиоцима (енг. executor), а они се деле у две породице. Прва одржава задати број виртуелних корисника, при чему сваки корисник шаље нови захтев тек пошто добије одговор на претходни; тиме стопа долазака зависи од брзине одговора сервера, што одговара затвореном моделу оптерећења. Друга породица одржава задату долазну стопу, односно број започетих пролаза у секунди, без обзира на то колико сервер брзо одговара, што одговара отвореном моделу. У другом случају број виртуелних корисника задаје се као резерва потребна да се стопа оствари, а ако резерва није довољна, алат бележи број неизведених пролаза као показатељ да остварена стопа заостаје за задатом [24].

Исправност одговора проверава се провером услова, који не прекида извршавање него води евиденцију о уделу успешних, док се прагови постављају над мереним величинама и одређују да ли је испитивање у целини успело; праг се може подесити и тако да извршавање прекине чим буде прекорачен. Резултати се приказују сажето по завршетку, али и прослеђују спољним системима у току самог извршавања, међу њима и систему Prometheus, путем удаљеног уписа.

4. Redirect Апликација

4.1. Намена и функционални захтеви

Апликација развијена за потребе овог рада представља систем за скраћивање и преусмеравање URL адреса. Корисник предаје дугачку адресу и добија кратак код, а приступ рути за преусмеравање са кратким кодом преусмерава посетиоца на изворну адресу. Свака посета се бележи, а забележени подаци су доступни. Скраћивање адреса представља једноставну и добро познату функционалност, али систем који је остварује има управо ону особину која је предмет испитивања: путања преусмеравања је кратка, извршава је сервис без стања, а свако извршавање захтева приступ заједничком слоју података. Тиме мерене разлике између конфигурација потичу од начина скалирања, а не од сложености пословне логике. Функционални захтеви постављени пред систем приказани су у табели (Табела 4.1.1.).

Захтев	**Сервис**
Стварање кратког кода на основу изворне адресе	Urlshortener
Трајно чување пара кратког кода и изворне адресе	Urlshortener
Преусмеравање посетиоца са кратког кода на изворну адресу	Redirect
Бележење сваке остварене посете	Analytics
Преглед забележених посета по кратком коду	Analytics

Табела 4.1.1. Функционални захтеви и сервиси који имплементирају захтеве

4.2. Архитектура система

Систем се састоји од три сервиса, две базе података и посредника за размену података. Сервиси су међусобно независни и извршавају се у посебним процесима. Компоненте система су:

1. Сервис Urlshortener задужен је за стварање кратких кодова. Прима изворну адресу, ствара запис о вези између кода и адресе и уписује га у базу података.

2. Сервис Redirect опслужује путању преусмеравања. За задати код проналази изворну адресу и посетиоцу враћа одговор са статусом 302.

3. Сервис Analytics задужен је за податке о посетама. Преузима поруке о забележеним посетама, уписује их у сопствену базу и излаже их кроз за то намењену руту.

4. База података о везама са називом urlshortener садржи табелу веза између кодова и изворних адреса. Деле је сервиси Urlshortener и Redirect, при чему први у њу уписује, а други из ње чита.

5. База података о посетама са називом analytics садржи табелу забележених посета и користи је искључиво сервис Analytics.

6. RabbitMQ, посредник за размену порука прима поруке о посетама од сервиса Redirect и прослеђује их сервису Analytics.

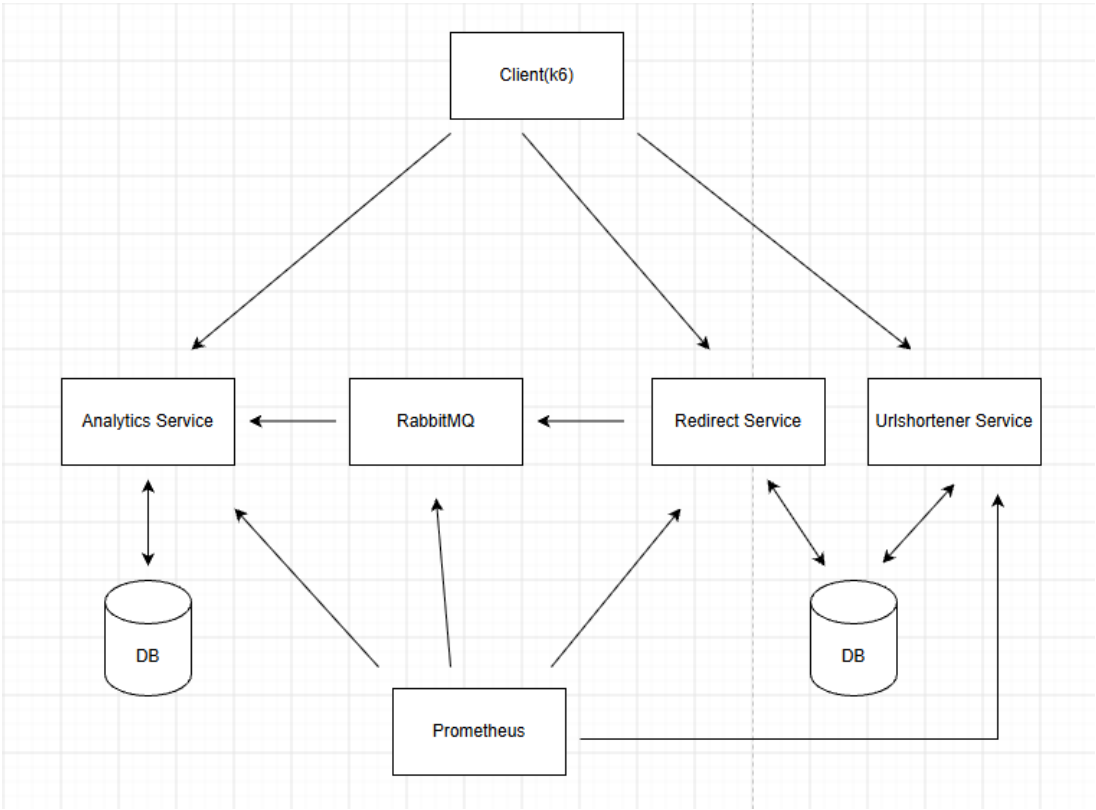
Обрада захтева одвија се на две путање:

1. Синхрона путања обухвата преусмеравање. Захтев стиже сервису Redirect, који податак о коду чита из базе и одмах враћа одговор
2. Асинхрона путања обухвата бележење посете. Сервис Redirect предаје податак о посети посреднику за размену порука и не чека његову обраду, док сервис Analytics поруку преузима и уписује је у своју базу

Раздвајање путања је намерно, будући да трајање уписа посете не сме да улази у време одзива преусмеравања. Сервиси Redirect и Urlshortener деле исту базу података. То одступа од начела да сваки сервис има сопствену базу. Одступање је свесно: оба сервиса раде над истим подацима и од истих података зависе, па њихово раздвајање не би било могуће без увођења додатне комуникације између сервиса при свакој посети. Функционално остају одвојени и независни, јер сваки задржава сопствену одговорност и сопствени процес. Такав распоред складишта у пракси није редак, а управо он и представља предмет испитивања у овом раду. Сваки сервис излаже два HTTP порта:

1. Јавни порт опслужује кориснике и преко њега пролази мерени саобраћај.
2. Управљачки порт служи оркестратору и систему за надзор, а излаже податке о стању сервиса и показатеље његовог рада.

Раздвајање портова обезбеђује да провере стања и прикупљање показатеља не оптерећују исти порт преко кога пролази мерени саобраћај, чиме тај саобраћај не улази у измерене вредности. На следећој слици (Слика 4.2.1.) је приказана архитектура система.



Слика 4.2.1. Архитектура redirect апликације

4.3. Сервиси

Сва три сервиса изграђена су по истом обрасцу. Написани су у програмском језику Go, користе исти оквир за опслуживање HTTP захтева и исти управљач везама према бази података, а унутрашња подела на слојеве иста је у сваком од њих. У наставку ће бити описани сервиси Urlshortener, Redirect и Analytics.

4.3.1. Urlshortener сервис

Urlshortener сервис омогућава да од задате изворне адресе изгенерше кратак код да сачува везу и да омогући увид у постојеће везе. Са становишта корисника, рад сервиса своди се на предају дугачке адресе и пријем кратког кода. Сервис ствара код који претходно не постоји, повезује га са задатом адресом и трајно чува ту везу. Од тог тренутка код је употребљив, односно сервис Redirect може на основу њега да пронађе изворну адресу. Друга крајња тачка омогућава да се за постојећи код провере подаци о вези, што служи провери исправности и није део мерене путање. Веза између кода и адресе чува се у бази података коју овај сервис дели са сервисом Redirect. Тиме је успостављен однос у коме један сервис у заједничку базу уписује а други из ње чита, што је управо особина система која се у овом раду испитује. Стварање кратког кода обавља се тако што се из криптографски безбедног извора случајних вредности узима низ бајтова, који се потом пресликава у скуп од шездесет два знака, састављен од малих и великих слова и цифара. Дужина кода задаје се као параметар. Пошто се код ствара насумично постоји мала вервоатноћа да се генерише већ постојећи код, а над колоном кода постоји јединствени индекс, поновљена вредност била би одбијена на нивоу базе података. Ако код већ постоји онда на одређену повратну вредност из слоја Repository поново генерише код у једној петљи до 100 пута. Скуп података над којим се експерименти изводе није створен преко овог сервиса, него уписан унапред, детерминистичким поступком описаним у одељку 5.2, чиме је обезбеђено да сва мерења полазе од истоветног садржаја базе.

4.3.2. Redirect сервис

Redirect сервис омогућава да посетилац који је приступио кратком коду буде преусмерен на изворну адресу, и да при том забележи да је посета остварена. Са становишта корисника, рад сервиса своди се на једну размену. Посетилац отвара кратку адресу, сервис на основу кода проналази изворну адресу и одговара упутством прегледачу да настави ка њој. Уколико код не постоји или је деактивиран, посетилац добија обавештење да тражени садржај није пронађен. Ниједан други корак не постоји, јер је реч о путањи која се у овом раду мери и која стога мора остати што краћа. Бележење посете сервис не обавља сам. Он податак о оствареној посети предаје на даљу обраду, а уписује га сервис Analytics. Таква подела уведена је како трајање уписа не би улазило у време одзива преусмеравања, о чему је реч у наставку. Има једну руту GET /:code , која не припада ниједној групи путања и не пролази кроз додатне слојеве усмеравања. Обрада захтева састоји се од једног упита над базом података, којим се за задати код проналази изворна адреса, уз услов да веза није деактивирана. Упит користи јединствени индекс над колоном кода, због чега се извршава као претраживање по индексу без пролаза кроз табелу. Када код постоји, враћа се одговор са статусом 302 и заглављем које забрањује привремено чување одговора, чиме се спречава да посредници одговоре на наредне захтеве уместо сервиса. Када код не постоји, враћа се статус 404, а у случају друге серверске грешке враћа се статус 500. Након што је одговор обликован, податак о посети се уписује у RabbitMQ queue. Уколико је queue пун порука се одбацује и забележи се одбачај. На овај начин предаја никад не блокира него је потпуно асинхрона. Цена овога да се одређени број посета може изгубити. Број успешних послатих порука и број одбачених порука се бележи.

4.3.3. Analytics сервис

Analytics сервис трајно чува податке о остварeним посетама и чини их доступним у сажетом облику. Уписивање се не обавља појединачно. Преузети подаци се групишу и уписују заједно, при чему се група затвара када достигне задату величину или када истекне задати временски размак, зависно од тога шта наступи прво. Читава група шаље се бази као једна серија наредби, чиме се број одлазака до базе смањује са једног по посети на један по групи. У систему у коме број посета вишеструко премашује број осталих операција, та разлика битно утиче на оптерећење базе. Преглед статистике доступан је преко крајње тачке GET /api/v1/analytics/:code. Одговор садржи укупан број забележених посета за задати код, број посета у последња двадесет четири часа и тренутак последње посете. Све три вредности добијају се једним упитом, уз ослањање на индекс над колоном кода и тренутком посете. Сервис користи сопствену базу података, одвојену од базе у којој се чувају везе између кодова и адреса, чиме учестало уписивање не оптерећује базу из које сервис Redirect чита на мереној путањи.

4.4. Модел података

Подаци система смештени су у две одвојене базе:

1. Прва садржи табелу веза између кратких кодова и изворних адреса, а користе је сервиси Urlshortener и Redirect.

2. Друга садржи табелу забележених посета и користи је искључиво сервис Analytics. Између двеју база не постоји веза на нивоу базе података. У следећој табели (Табела 4.4.1.) је приказана табела веза.

Колона	Тип	Опис
id	BIGSERIAL	Примарни кључ
code	VARCHAR(30)	Кратак код, јединствен на нивоу табеле
target_url	TEXT	Изворна адреса
expires_at	TIMESTAMPTZ	Тренутак истека важења везе
is_active	BOOLEAN	Ознака да ли је веза активна
created_at	TIMESTAMPTZ	Тренутак стврана записа

Табела 4.4.1. Структура табеле веза

Над колоном кода постављен је јединствени индекс. Он истовремено обавља две улоге: спречава да две везе добију исти код и представља приступни пут за упит који се извршава на мереној путањи. Пошто се тај упит извршава при сваком преусмеравању, његово понашање непосредно одређује оптерећење базе података, због чега постоји индекс. У следећој таблеи (Табела 4.4.2.) је приказана табела посета.

Колона	Тип	Опис
id	BIGSERIAL	Примарни кључ
code	VARCHAR(30)	Кратак код, јединствен на нивоу табеле
occurped_at	TIMESTAMPTZ	Тренутак остварења посете
referrer	TEXT	Адреса са које је посетилац дошао
user_agent	TEXT	Ознака прегледача посетиоца

Табела 4.4.2. Структура табеле посета

Између колоне кода у табели посета и колоне кода у табели веза не постоји ограничење референцијалног интегритета. Разлог је физичка одвојеност база, због које провера постојања кода при упису посете није изводљива у оквиру једне трансакције. Последица је да табела посета може садржати редове са кодом који у међувремену више не постоји.

4.5. Комуникација између сервиса

У систему постоје две врсте комуникације:

1. Синхрона комуникација обухвата позиве које корисник упућује сервисима преко протокола HTTP, описане у одељку 4.3. Позивалац чека одговор и добија га у оквиру истог захтева. Сервиси међусобно не остварују синхроне позиве, па отказ или успоравање једног сервиса не блокира обраду у другом.

2. Асинхрона комуникација обухвата размену порука између сервиса Redirect и Analytics и одвија се преко посредника за размену порука. Када сервис Redirect обради преусмеравање, податак о оствареној посети не уписује сам, него га предаје посреднику и одмах наставља рад. Посредник поруку чува све док је сервис Analytics не преузме и упише у своју базу података. Пошиљалац и прималац тако не морају да раде истовремено нити истом брзином: уколико сервис Analytics привремено заостане, поруке се задржавају код посредника и биће обрађене касније. Пошто сервис Redirect не чека потврду да је порука обрађена, трајање уписа не улази у време одзива преусмеравања.

4.6. Инструментација

Апликација у овом раду има улогу мерног инструмента, због чега показатељи њеног рада нису накнадни додатак него део њеног обликовања. Сваки сервис излаже сопствене показатеље у облику који систем за надзор може непосредно да преузме. Метрике се излажу кроз REST API рута, одвојеном од оног преко кога пролази мерени саобраћај, на путањи /metrics. У следећој табели су приказане све метрике доступне кроз апликацију (Табела 4.6.1.).

Група	Назив	Шта бележи
HTTP	http_requests_total	број захтева, разврстан по статусу одговора
	http_request_duration_seconds	расподела трајања обраде захтева
	http_requests_in_flight	број захтева који се обрађују у датом тренутку
База података	db_queries_total	број упита, разврстан по исходу
	db_query_duration_seconds	расподела трајања извршавања упита
	db_pool_max_conns	подешена највећа величина скупа конекција
	db_pool_total_conns	број тренутно отворених конекција
	db_pool_acquired_conns	број конекција које су у датом тренутку заузете
	db_pool_empty_acquire_total	број покушаја заузимања при коме ниједна конекција није била слободна
	db_pool_empty_acquire_wait_seconds_total	укупно време проведено у чекању на слободну конекцију
RabbitMQ	clicks_published_total	број успешно послатих порука кроз Redirect сервис
	clicks_dropped_total	број одбачених порука кроз Redirect сервис
	clicks_buffer_used, clicks_buffer_capacity	попуњеност бафера кроз Redirect сервис
	clicks_consumed_total	број преузетих порука, по исходу Analytics сервиса
	clicks_flush_total	број уписаних група, по исходу Analytics сервиса
	clicks_flush_duration_seconds	расподела трајања уписа једне групе Analytics сервиса
	clicks_batch_size	расподела величине уписаних група Analytics сервиса

Табела 4.6.1. Метрике доступне у сервисима

4.7. Контејнеризација

Сваки сервис има засебан Dockerfile. Сваки сервис припрема се за извршавање у облику слике контејнера. Слика садржи извршни програм и све што му је потребно за рад, због чега се сервис понаша истоветно без обзира на то где се извршава. Сва три сервиса користе истоветан опис градње. Пошто су остварени у истом програмском језику и по истом обрасцу, ниједном од њих није потребно посебно подешавање, што уједно смањује могућност да се сервиси разликују по нечему што није део њихове улоге. У следећом лиситингу (Листинг 4.7.1.) је приказан Dockerfile.

```
FROM golang:1.25-alpine AS build

WORKDIR /src

COPY go.mod go.sum ./

RUN go mod download

COPY . .

RUN --mount=type=cache,target=/go/pkg/mod \

   --mount=type=cache,target=/root/.cache/go-build \

   CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \

   go build -ldflags="-s -w" -o /out/ ./cmd/...

FROM gcr.io/distroless/static-debian12:nonroot

COPY --from=build /out/ /app/

USER nonroot:nonroot

ENTRYPOINT ["/app/api"]
```

Листинг 4.7.1. Опис Dockerfile-а

4.8. Распоређивање на Azure Kubernetes Service

Систем се распоређује на управљани Kubernetes кластер у оквиру услуге Azure Kubernetes Service. Сви објекти смештени су у засебан именски простор, чиме су одвојени од системских компоненти кластера и од евентуалних других апликација. Начин замене примерака при испоруци нове верзије подешен је тако да се нови примерак покреће тек пошто стари буде уклоњен. Уобичајено подешавање, по коме се нови примерак покреће упоредо са старим, у овом систему не би било изводљиво: правило распоређивања описано у наставку допушта највише један примерак по чвору, па за додатни примерак не би постојао слободан чвор и испорука би остала недовршена. Приступ примерцима остварује се преко објеката сервиса. Сваки такав објекат пружа постојану адресу унутар кластера и расподељује долазни саобраћај међу свим примерцима који му припадају. Ниједан сервис није изложен изван кластера, будући да се и генератор оптерећења извршава унутар њега. Тиме је из мерене путање уклоњен спољашњи расподељивач оптерећења, који би иначе унео сопствено кашњење и сопствено ограничење капацитета. Сваки ресурс описан је засебном датотеком, а датотеке су именоване бројчаним префиксом који одређује редослед примене. Тим редоследом ресурси од којих други зависе настају први: именски простор пре свега осталог, подешавања и тајне пре примерака који их користе, а измене шеме базе података пре покретања сервиса.

Сервис Urlshortener описан је са пет датотека:

- 10-configmap.yaml — ConfigMap
- 11-secret.yaml — Secret
- 12-migrate-job.yaml — Job

- 13-deployment.yaml — Deployment
- 14-service.yaml — Service

Сервис Redirect описан је са четири датотеке:

- 20-configmap.yaml — ConfigMap
- 21-secret.yaml — Secret
- 22-deployment.yaml — Deployment
- 23-service.yaml — Service

Сервис Analytics описан је са пет датотека:

- 30-configmap.yaml — ConfigMap
- 31-secret.yaml — Secret
- 32-migrate-job.yaml — Job
- 33-deployment.yaml — Deployment
- 34-service.yaml — Service

Сервис Redirect једини нема посао за измену шеме базе података, будући да не поседује сопствену базу него чита из базе коју одржава сервис Urlshortener. Поред микросервиса, у кластеру се распоређују и компоненте које чине окружење: базе података, посредник за размену порука и систем за прикупљање и приказ показатеља. Посредник за размену порука описан је датотекама 40-deployment.yaml (Deployment) и 41-service.yaml (Service).

Систем за прикупљање показатеља описан је са четири датотеке:

- 50-prometheus-rbac.yaml — ServiceAccount, ClusterRole и ClusterRoleBinding
- 51-prometheus-config.yaml — ConfigMap
- 52-prometheus-deployment.yaml — PersistentVolumeClaim и Deployment
- 53-prometheus-service.yaml — Service

Систем за приказ показатеља описан је са три датотеке:

- 54-grafana-config.yaml — ConfigMap
- 55-grafana-deployment.yaml — PersistentVolumeClaim и Deployment
- 56-grafana-service.yaml — Service

Генератор оптерећења описан је једном датотеком, 57-k6-deployment.yaml (Deployment). База података коју деле сервиси Urlshortener и Redirect описана је датотекама 60-pvc.yaml (PersistentVolumeClaim), 61-deployment.yaml (Deployment) и 62-service.yaml (Service).

База сервиса Analytics описана је на истоветан начин, датотекама 70-pvc.yaml, 71-deployment.yaml и 72-service.yaml.

Систему за прикупљање показатеља потребна су додатна права, будући да мора да пронађе подове и чворове у кластеру како би с њих преузимао показатеље. Та права додељују му се засебним ресурсима, чиме је приступ ограничен на само оно што му је за прикупљање неопходно.

Генератор оптерећења распоређује се као примерак који не обавља посао док не буде покренут. Скрипта се у њега преноси непосредно пре мерења, чиме исти примерак служи свим експериментима и не мора се поново покретати између појединачних мерења.

4.9. Групе чворова

Чворови кластера нису јединствени скуп него су подељени у пет група. Разлог за поделу произлази из захтева мерења: свака компонента система треба да се извршава на чвору који не дели ни са једном другом мереном компонентом, па потрошња ресурса измерена на нивоу чвора припада тачно једној компоненти. Подела уз то омогућава да се величина чворова мења за једну компоненту, а да остале остану непромењене. Састав група дат је у табели (Табела 4.9.1.).

Група	**Ознака**	**Тип чвора**	**Број чворова**	**Компоненте**
Системска	system	Standard_D4s_v5	1	Prometheus, Grafana
Помоћна	support	Standard_D2s_v5	3	RabbitMQ, Urlshortener, Analytics
Апликативна	app	променљива	променљива	Redirect
За базе података	db	Standard_D4s_v5	2	обе инстанце базе
За генератор оптерећења	loadgen	Standard_D4s_v5	1	k6

Табела 4.9.1. Групе чворова

Свака група носи ознаку којом се чворови у њој разликују од осталих. Апликативна група је једина чија се величина и број чворова мењају током експеримената, будући да у њој ради једини сервис који се скалира. Аутоматско прилагођавање броја чворова искључено је у свим групама. Уколико би било укључено, кластер би сам додавао или уклањао чворове при промени оптерећења, чиме би се конфигурација мењала усред мерења. Правило да се на једном чвору налази највише један под има непосредну последицу по величину апликативне групе, број чворова у њој мора бити једнак броју примерака сервиса Redirect.

5. Методологија мерења

5.1. Променљиве и контролисани фактори

Резултат мерења може се протумачити само уколико је познато шта се између два мерења променило, а шта је остало исто. Када се истовремено промени више околности, измерена разлика не може се поуздано приписати ниједној од њих. Због тога се пре извођења експеримената одређују променљиве и контролисани фактори. Разликују се три врсте величина: оне које се намерно мењају, оне које се мере, и оне које се држе непроменљивим како измерене разлике не би потицале од њих. Независне променљиве раздвојене су на два нивоа. Унутар једног покретања мења се једино стопа долазака захтева, која се повећава степенасто. Између покретања мења се конфигурација извршног окружења, и то у сваком експерименту само по један њен чинилац. Такво раздвајање омогућава да се за сваку конфигурацију одреди њена крива понашања под растућим оптерећењем, а затим да се те криве међусобно упореде. Зависне променљиве обухватају величине које описују понашање система под задатим оптерећењем. У следећој таблеи (Табела 5.1.1.) приказују се независне порменљиве.

Величина	**Где се мења односно мери**
Стопа долазака захтева	Сви експерименти
Однос врста захтева	Експерименти 1 и 2
Број инстанци сервиса Redirect	Експеримент 3
Величина чвора сервиса	Експеримент 4
Величина чвора базе	Експеримент 5

Табела 5.1.1. Независне променљиве

У следећој табели (Табела 5.1.2.) приказане се зависне променљиве.

Величина	**Где се мења односно мери**
Капацитет	Сви експерименти
Пропусност	Сви експерименти
Кашњење са стране клијента	Сви експерименти
Удео неуспешних захтева	Сви експерименти
Утрошено процесорско време	Сви експерименти
Заузеће радне меморије	Сви експерименти
Искоришћеност скупа конекција	Експерименти 3, 4, и 5
Време извршавања упита над базом података	Сви експерименти

Табела 5.1.2. Зависне променљиве

Контролисани фактори су они чиниоци који могу утицати на резултат, а који се намерно држе непроменљивим. Њихове вредности наведене су у табели (Табела 5.1.3.). Скуп

података је истоветан у сваком покретању и ствара се детерминистичким поступком описаним у одељку 5.2., чиме је искључена могућност да разлике потичу од садржаја базе. Врста оптерећења у експериментима од трећег до петог сведена је искључиво на преусмеравање, чиме се избегава увођење другог уског грла на путањи уписа и обезбеђује да скуп података током мерења остане непромењен.

Фактор	**Вредност**
Скуп података	500.000 веза, стваран детерминистички
Величина скупа конекција по примерку	2(Експерименти 3,4,5) , 10(Ескперименти 1,2)
Врста оптерећења у експериментима 3–5	искључиво преусмеравање
Чвор базе података у експериментима 1, 2, 3 и 4	D4s_v5
Број примерака посредника за размену порука	1
Породица коришћених виртуелних машина	Dsv5
Аутоматско скалирање кластера	искључено
Аутоматска надоградња платформе и чворова	искључено
Генератор оптерећења	засебан чвор, не дели ресурсе са мереним системом
Верзија слике контејнера	непромењена кроз све експерименте
Почетно стање пре сваког покретања	успостављено поступком из одељка 5.6.

Табела 5.1.3. Контролисани фактори

Породица виртуелних машина одабрана је тако да обухвата целу лествицу величина које се у експериментима користе, чиме је обезбеђено да се при промени величине чвора мења само количина ресурса, а не и врста хардвера на коме се мерење обавља. Из истог разлога избегнуте су машине са променљивим процесорским учинком, будући да оне током дужег мерења мењају расположиву снагу и чине резултате непоновљивим.

5.2. Скуп података и његово генерисање

Сва мерења изведена су над истоветним скупом података, који чини пет стотина хиљада веза између кратког кода и изворне адресе. Скуп је уписан у базу пре почетка мерења и током експеримената се не мења, будући да оптерећење у експериментима од трећег до петог обухвата искључиво преусмеравање, које не уписује податке. Скуп није створен коришћењем самог сервиса Urlshortener, него посебном наредбом над базом података, приказаном у листингу (Листинг 5.2.1.). Кодови се добијају скраћивањем хексадецималног сажетка редног броја на дванаест знакова, а изворне адресе садрже исти редни број. Пошто се обе вредности изводе из редног броја, поновно извршавање наредбе даје потпуно исти садржајбазе.

```
INSERT INTO links (code, target_url)
SELECT substr(md5(i::text), 1, 12),
    'https://example.com/page/' || i
FROM generate_series(1, 500000) AS i
WHERE NOT EXISTS (SELECT 1 FROM links);
```

Листинг 5.2.1. Наредба за стварање скупа података

Разлог за такав поступак јесте поновљивост. Сервис Urlshortener ствара кодове из криптографски безбедног извора случајних вредности, који се по својој намени не може подесити тако да даје унапред одређени низ. Скуп података створен преко сервиса стога не би могао бити тачно поновљен, што би онемогућило поређење мерења изведених у различитим приликама. Детерминистички поступак ту тешкоћу отклања: скуп се може обрисати и поново створити, а садржај базе биће једнак до последњег бајта. Генератор оптерећења не бира кодове насумично из целог простора могућих вредности, него из списка кодова који стварно постоје у бази. Списак је извезен из базе након уписа, садржи свих пет стотина хиљада кодова и заузима око осам мегабајта. Учитава се једном при покретању и дели међу свим виртуелним корисницима, чиме се избегава вишеструко заузимање меморије. Захваљујући томе сваки упућени захтев погађа постојећи код и пролази целу путању обраде, па измерено кашњење не садржи јефтине одговоре о непронађеном ресурсу. Избор кода при сваком захтеву обавља се равномерном расподелом над целим списком.

5.3. Модел оптерећења

Оптерећење се задаје према отвореном моделу, изложеном у одељку 2.5.2. Генератор упућује захтеве према унапред задатој стопи, независно од тога колико систему треба да одговори, чиме измерено кашњење остаје веродостојно и у тренутку када систем почне да заостаје, све док генератор успева да упути задату стопу. На степенима на којима то више не успева, захтеви које није упутио немају забележено трајање, па приказано кашњење представља доњу процену стварног. Стопа долазака је, дакле, задата величина, а не последица понашања система.

Профил оптерећења обликован је као степенасто повећање стопе. Свако покретање почиње загревањем, након кога следи низ степена, при чему сваки степен обухвата постепено повећање до задате стопе и потом задржавање на тој стопи. Загревање се одбацује из мерења.

Мерење не обухвата цео степен него само његов завршни део. Прозор мерења траје седамдесет пет секунди и завршава се на крају задржавања. Тиме је обухваћено искључиво устаљено стање, док прелазни део, у коме се стопа још повећава, остаје изван мерења.

Дужина самог прозора одабрана је тако да обухвати око пет узорака показатеља о потрошњи ресурса добиијеног од cAdvisor-a, што је најмањи број из кога се стопа промене уопште може израчунати. Мала густина узорковања последица је интервала којим оркестратор прикупља податке о контејнерима, који се на коришћеној платформи не може подесити.

У прва два експеримента сва три сервиса оптерећују се истовремено, при чему се у првом свакоме упућује једнака стопа, а у другом стопе стоје у односу који одговара очекиваној употреби. У преостала три експеримента оптерећење обухвата искључиво преусмеравање, то јест сервис Redirect.

У експериментима 3,4 и 5 примењен је исти низ стопа четири, осам, четрнаест и двадесет две хиљаде захтева у секунди. Јединствен низ омогућава да се понашање конфигурација упореди при истом испорученом оптерећењу, а не само у тачки засићења сваке од њих. Низ је постављен тако да најнижи степен све конфигурације опслуже без напрезања, а да највиши превазиђе капацитет оних са најмањим бројем расположивих веза.

5.4. Дефиниција капацитета и критеријум засићења

Капацитет конфигурације одређује се као највиша стопа захтева коју она опслужује без нарушавања задатих услова. Пошто се оптерећење задаје у степенима, капацитет одговара

највишем степену који је конфигурација издржала, при чему су и сви претходни степени морали бити издржани. Стварна вредност лежи између тог степена и првог који није издржан, па наведени капацитет представља њену доњу процену.

Деведесет пети перцентил узима се из хистограма који сервис сам води, а не из вредности које доставља генератор оптерећења. Генератор доставља статистике накупљене од почетка покретања, па на вишим степенима његова вредност садржи и раније степене, изведене при нижем оптерећењу, и приказује време одговора мањим него што у посматраном степену јесте. Дефнисана у коду приказана у следећем листингу (Лситинг 5.4.1.).

```
RequestDuration = promauto.NewHistogram(prometheus.HistogramOpts{
        Name:    „http_request_duration_seconds“,
        Help:    „Handler latency.“,
        Buckets: []float64{0.0005, 0.001, 0.0025, 0.005, 0.01, 0.025, 0.05, 0.1, 0.2, 0.25, 0.5, 1, 2.5,
5, 10},
})
```

Листинг 5.4.1. Дефиниција хистограма времена извршавања захтева

Вредност перцентила добија се проценом положаја унутар разреда хистограма и не представља тачно измерено време. На исход поређења то не утиче, будући да су све измерене вредности или реда једне милисекунде или веће од четири стотине милисекунди, дакле далеко од прага са обе стране.

Услови се не проверавају током самог покретања, него накнадно, над прикупљеним вредностима. Праг задат генератору оптерећења износи пет секунди и служи искључиво да заустави покретање када конфигурација престане да одговара; он није критеријум засићења. Свака конфигурација која неки степен није издржала отказала је тако што није испоручила задату стопу. Услов се ипак задржава, будући да у отвореном моделу оптерећења систем може да прима сав задати саобраћај док му ред захтева расте, при чему испоручена стопа остаје тачна а време одговора се повећава.

5.5. Мерење величина и начин прикупљања

Подаци се прикупљају из три извора: сервиси, cAdvisor и k6 алат. Сваки сервис има руту /metrics која се пулује са Prometheus алатом. Подаци о потрошњи ресурса преузимају се са сваког чвора кластера преко cAdvisor, две најбитније метрике које излаже су коришћеност меморије и коришћеност процесора. Генератор k6 своје податке не излаже него их сам шаље систему за надзор, будући да се извршава само током мерења.

Прикупљање метрика обавља посебна скрипта. Она садржи четрдесет три именована упита. За сваку меру упућује један упит ка Prometheus алату и добија једну вредност по компоненти, чиме се цео прозор сажима у један број. Резултати се уписују у датотеку у којој сваки ред садржи ознаку покретања, назив мере, назив компоненте и измерену вредност. Пошто се прозор и завршни тренутак задају при позиву, иста скрипта користи се у свим експериментима.

Учесталост прикупљања разликује се по изворима. Показатеље које излажу сервиси и посредник за размену порука систем за надзор преузима на сваких пет секунди. Податке о потрошњи ресурса преузима на сваких десет секунди, али је стварна временска разлучивост тих података око петнаест секунди, будући да их сам чвор освежава ређе него што се преузимају, па се поновљене вредности одбацују. Генератор оптерећења своје податке шаље на сваких пет секунди. Из тога следи да прозор мерења од седамдесет пет секунди садржи око петнаест узорака показатеља које излажу сервиси и око пет узорака показатеља о потрошњи ресурса.

Одговор који систем за надзор враћа на један упит садржи по једну вредност за сваку компоненту на коју се тај упит односи. Скрипта сваку такву вредност претвара у по један ред и додаје је у датотеку. Упит за пропусност тако даје три реда, по један за сваки сервис, док упит за потрошњу процесорског времена даје девет, по један за сваку компоненту система. Пошто се поступак понавља за свих четрдесет три упита, једно прикупљање производи око стотину педесет редова.

Подаци о потрошњи ресурса не долазе од самих сервиса него од система за надзор контејнера, који је уграђен у сваки чвор кластера. Тај систем прати сваки контејнер који се на чвору извршава и излаже податке о његовој потрошњи, без обзира на то да ли је контејнер уопште припремљен за надгледање. Захваљујући томе обухваћене су и компоненте које саме не излажу показатеље, попут база података и посредника за размену порука.

За процесорско време користи се бројач утрошених процесорских секунди, назив у cAdvisor-у је container_cpu_usage_seconds_total. Реч је о величини која само расте и која показује колико је укупно процесорског времена контејнер потрошио од свог покретања. Сама по себи та вредност није употребљива за поређење, будући да зависи од тога колико контејнер ради. Употребљива постаје тек када се посматра њен пораст током прозора мерења: подељен трајањем прозора, тај пораст даје просечан број заузетих језгара. Па се добија вредност потрошених процесорских секунди по реалној секунди времена. Утрошено процесорско време у раду се изражава у милијезгрима. Једно милијезгро одговара једној милисекунди процесорског времена утрошеној током једне секунде стварног времена, односно хиљадитом делу једног језгра.

За радну меморију користи се мера радног скупа, назив у cAdvisor-у је container_memory_working_set_bytes. За разлику од процесорског времена, које се бележи бројачем који само расте, радни скуп представља тренутно стање — колико меморије контејнер држи у датом тренутку. Због тога се на њега не примењује рачунање пораста, него се узима његова просечна вредност током прозора мерења. Радни скуп обухвата меморију коју процес заиста држи и не може је ослободити без последица. У њега не улазе странице датотечног система које оперативни систем може да одбаци под притиском, јер оне нису потрошња саме апликације. Вредност се преузима у бајтовима и дели са бројем бајтова у мебибајтима. Јединица је мебибајт (MiB) и представља износ, а не брзину.

Расположиво процесорско време по чвору не долази из система за надзор, него се очитава из описа чвора и бележи уз свако покретање. Оно је потребно да би се утрошено процесорско време могло изразити као удео капацитета чвора, што се користи у петом експерименту.

Оптерећење ствара извршилац ramping-arrival-rate алата k6, који припада породици извршилаца са задатом долазном стопом. Генератор на почетку располаже са 1.600 виртуелних корисника и по потреби додаје нове, до највише 8.000. Док има слободних виртуелних корисника, стопа долазака не зависи од времена одговора сервиса. Када су сви заузети, генератор не може да започне нове захтеве, па их не шаље, већ их бележи као одбачене. На таквим степенима број истовремених захтева ограничен је на 8.000, генератор се понаша као затворени модел са 8.000 корисника, а обрађена стопа показује колико захтева сервис може да обради.

5.6. Ток једног мерења

Мерење се обавља по утврђеном редоследу, како би свако покретање полазило од истоветног стања система. Поступак се дели на припрему конфигурације, која се обавља

једном по конфигурацији, и припрему покретања, која се обавља пре сваког појединачног мерења.

Припрема конфигурације подразумева да се група чворова намењена апликацији поново ствара са траженом величином и бројем чворова, а број примерака сервиса поставља се на тражену вредност. Након тога се проверава да је сваки примерак заиста распоређен на засебан чвор и да су подешавања преузета из измењене конфигурације.

Припрема покретања подразумева да се сервиси поново покрећу. Поново се покрећу Redirect сервис, Analytics сервис, Urlshortener сервис и k6 под. Затим се над базом података изврше следеће радње:

1. Обришу се сви редови изнад полазног скупа

2. Над табелом се извршава наредба VACUUM FULL ANALYZE, којом се ослобађа простор који су заузели обрисани редови и истовремено освежава статистика на основу које планер бира начин извршавања упита
3. Табела и припадајући индекс учитавају се у меморију базе наредбом pg_prewarm, чиме свако покретање почиње са истоветним садржајем привременог складишта и уклања се утицај приступа диску
4. Табела посета се празни наредбом TRUNCATE, и позива се наредба CHECKPOINT

Спровођење подразумева упис скрипте захтева у k6 алат, покреће се генератор и бележи се тренутак почетка.

Прикупљање подразумева да се по истеку сваког степена прикупљају се вредности из прозора који се завршава на његовом крају.

Чување подразумева да се резултати сваког покретања уписују се у засебну датотеку.

6. Експерименти

Свих пет експеримената изведени су у три понављања. Резултати сваког покретања чувају се у засебној датотеци, именованој према ознаци експеримента, ознаци конфигурације и редном броју понављања, а датотеке су разврстане у именике по истом начелу. Тиме је свака вредност наведена у наставку овог поглавља повезива са изворним подацима из којих је добијена.

6.1. Експеримент 1

Шаље се исти број захтева на сва три сервиса у три степена. Сва три сервиса имају једну реплику, свака од тих реплика је на свом засебном чвору. Чворови су на виртуелним машинама типа D2s_v5. Оптерећење је у три степена, при сто, две стотине и три стотине захтева у секунди према сваком сервису.

Експеримент је изведен три пута. Утрошено процесорско време по компоненти приказано је у табели (Табела 6.1.1.), а заузеће радне меморије у табели (Табела 6.1.2.). У обе табеле наведена је средња вредност трију покретања и стандардна девијација узорка, која показује поновљивост мерења: колико се вредности добијене у поновљеним покретањима разликују од средње вредности.

Компонента	100 захтева/s		200 захтева/s		300 захтева/s	
	СВ[mCPU]	Дев.[mCPU]	СВ[mCPU]	Дев.[mCPU]	СВ[mCPU]	Дев.[mCPU]
Сервис Redirect	26,56	0,38	42,88	0,71	61,72	0,26
Сервис Urlshortener	28,50	3,50	48,25	3,19	64,77	4,57
Сервис Analytics	37,47	3,42	60,88	1,81	85,64	5,53

Табела 6.1.1. Утрошено процесорско време по компоненти, у милијезгрима (средња вредност и стандардна девијација)

Компонента	100 захтева/s		200 захтева/s		300 захтева/s	
	СВ[MiB]	Дев.[MiB]	СВ[MiB]	Дев.[MiB]	СВ[MiB]	Дев.[MiB]
Сервис Redirect	10,37	0,01	9,99	0,06	9,91	0,26
Сервис Urlshortener	9,94	1,03	9,87	0,40	10,52	0,09
Сервис Analytics	9,66	0,31	9,00	0,16	8,92	0,28

Табела 6.1.2. Заузеће радне меморије по компоненти, у мебибајтима (средња вредност и стандардна девијација)

6.2. Експеримент 2

У стварној употреби система за скраћивање адреса број преусмеравања вишеструко премашује број стварања нових веза и број прегледа статистике. Сходно томе овај експеримент прикузаје перформансе апликције при реалистичним зехтевима. Оптерећење је у три степена, при укупно хиљаду, две хиљаде и три хиљаде захтева у секунди.

Експеримент је изведен три пута. Утрошено процесорско време по компоненти приказано је у табели (Табела 6.2.1.), а заузеће радне меморије у табели (Табела 6.2.2.). У обе табеле наведена је средња вредност трију покретања и стандардна девијација узорка, која

показује поновљивост мерења: колико се вредности добијене у поновљеним покретањима разликују од средње вредности. Захтеви су између сервиса расподељени у односу 99 : 0,9 : 0,1 одсто, што при укупним стопама од 1000, 2000 и 3000 захтева у секунди даје 990, 1980 и 2970 захтева у секунди ка сервису Redirect, 9, 18 и 27 захтева у секунди ка сервису Urlshortener и 1, 2 и 3 захтева у секунди ка сервису Analytics.

Компонента	1.000 захтева/s (990/9/1)		2.000 захтева/s (1.980/18/2)		3.000 захтева/s (2.970/27/3)	
	СВ[mCPU]	Дев.[mCPU]	СВ[mCPU]	Дев.[mCPU]	СВ[mCPU]	Дев.[mCPU]
Сервис Redirect	178,46	2,56	320,80	6,13	422,79	6,03
Сервис Urlshortener	5,61	0,12	10,04	0,70	14,17	0,43
Сервис Analytics	64,32	0,10	113,50	8,41	150,49	13,55

Табела 6.2.1. Утрошено процесорско време по компоненти, у милијезгрима (средња вредност и стандардна девијација)

Компонента	1.000 захтева/s (990/9/1)		2.000 захтева/s (1.980/18/2)		3.000 захтева/s (2.970/27/3)	
	СВ[MiB]	Дев.[MiB]	СВ[MiB]	Дев.[MiB]	СВ[MiB]	Дев.[MiB]
Сервис Redirect	10,29	0,33	10,27	0,08	10,31	0,10
Сервис Urlshortener	11,68	0,55	12,02	0,50	12,36	0,53
Сервис Analytics	8,07	0,08	8,59	0,04	8,69	0,36

Табела 6.2.2. Заузеће радне меморије по компоненти, у мебибајтима (средња вредност и стандардна девијација)

6.3. Експеримент 3

У овом експерименту величина чворова је иста у свакој конигурацији, број реплика се повећава а број конекција по једној реплици се не мења. Оптерећење је степенасто повећање стопе у четири степена. Величина скупа конекција по примерку износи две. Пошто сваки примерак држи сопствени скуп, укупан број веза ка бази расте са бројем примерака и износи четири, осам, односно шеснаест. Све три конфигурације подвргнуте су истом низу оптерећења: 4.000, 8.000, 14.000 и 22.000 захтева у секунди, како би се њихово понашање могло упоредити при једнаком оптерећењу, а не само у тачки засићења. Сваки степен траје деведесет секунди, при чему се мери прозор од седамдесет пет секунди. Свакој конфигурацији одговара три покретања. У следећој табели (Табела 6.3.1.) је приказана конфигурација експеримента.

Ознака	Број реплика	Чвор	Укупно [vCPU]	Број конекција према бази података
H4	2	D2s_v5	4	4
H8	4	D2s_v5	8	8
H16	8	D2s_v5	16	16

Табела 6.3.1. Конфигурације 3-ег експеримента

У следећој табели (Табела 6.3.2.) су приказани време одговора и обрађени захтеви.

Мера	Задата стопа [захтева/s]	H4		H8		H16	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
Време одговора, 95. перцентил [ms]	4.000	0,66	0,09	0,52	0,05	0,59	0,18
	8.000	2,14	1,69	0,64	0,08	0,60	0,19
	14.000	536,11	411,63	1,63	0,16	0,61	0,20
	22.000	943,26	33,57	212,61	240,54	1,06	0,60
Обрађена стопа [захтева/s]	4.000	4.000	0	4.000	0	4.000	0
	8.000	8.000	0	8.000	0	8.000	0
	14.000	12.395	2.650	14.000	1	14.000	0
	22.000	12.867	3.212	20.740	2.156	22.000	0

Табела 6.3.2. Време одговора и обрађена стопа сервиса Redirect (средња вредност и стандардна девијација)

У следећој табели (Табела 6.3.3.) су приказани чекање на конекцију према бази података по обрађеном захтеву и удео захтева који су чекали.

Мера	Задата стопа [захтева/s]	H4		H8		H16	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
Чекање по обрађеном захтеву [ms]	4.000	0,022	0,010	0,008	0,004	0,004	0,003
	8.000	0,295	0,243	0,016	0,004	0,009	0,008
	14.000	359,827	436,349	0,346	0,088	0,021	0,016
	22.000	640,046	188,303	151,894	222,680	0,213	0,120
Захтеви који су чекали [%]	4.000	8,3	1,7	3,1	0,9	1,4	0,8
	8.000	32,6	5,3	9,3	1,6	4,0	2,5
	14.000	85,1	25,9	26,8	1,4	5,7	2,3
	22.000	100,0	0,0	81,3	17,0	13,5	3,8

Табела 6.3.3. Чекање на конекцију према бази података (средња вредност и стандардна девијација)

У следећој табели (Табела 6.3.4.) је приказано утрошено процесорско време сервиса Redirect.

Мера	Задата стопа [захтева/s]	H4		H8		H16	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
По реплици [језгра]	4.000	0,36	0,02	0,19	0,02	0,10	0,01
	8.000	0,61	0,06	0,36	0,03	0,18	0,01
	14.000	0,87	0,08	0,57	0,04	0,30	0,01
	22.000	0,90	0,11	0,75	0,03	0,44	0,02
По реплици, искоришћеност процесора чвора [%]	4.000	18,7	1,2	10,1	0,9	5,2	0,3
	8.000	32,0	3,2	19,1	1,5	9,5	0,5
	14.000	45,7	4,3	29,8	1,9	15,9	0,6
	22.000	47,3	5,8	39,7	1,8	23,2	1,1
Укупно [језгра]	4.000	0,71	0,04	0,77	0,07	0,80	0,05
	8.000	1,22	0,12	1,45	0,11	1,45	0,07
	14.000	1,74	0,16	2,26	0,14	2,41	0,09
	22.000	1,80	0,22	3,01	0,14	3,52	0,16

Табела 6.3.4. Утрошено процесорско време сервиса Redirect (средња вредност и стандардна девијација)

У следећој табели (Табела 6.3.5.) је приказано заузеће радне меморије сервиса Redirect.

Мера	Задата стопа [захтева/s]	H4		H8		H16	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
По реплици [MiB]	4.000	33,22	0,21	19,73	0,08	13,35	0,19
	8.000	33,19	0,08	19,84	0,15	13,30	0,05
	14.000	104,05	84,63	19,99	0,17	13,23	0,09
	22.000	198,60	1,56	48,76	43,98	13,30	0,10
По реплици, од ограничења 256 MiB [%]	4.000	12,98	0,08	7,71	0,03	5,21	0,07
	8.000	12,96	0,03	7,75	0,06	5,19	0,02
	14.000	40,64	33,06	7,81	0,07	5,17	0,03
	22.000	77,58	0,61	19,05	17,18	5,19	0,04
Укупно [MiB]	4.000	66,44	0,42	78,92	0,33	106,80	1,50
	8.000	66,38	0,15	79,36	0,59	106,38	0,44
	14.000	208,10	169,27	79,97	0,68	105,81	0,71
	22.000	397,21	3,11	195,05	175,92	106,38	0,81

Табела 6.3.5. Заузеће радне меморије сервиса Redirect (средња вредност и стандардна девијација)

6.4. Експеримент 4

У овом експерименту број реплика износи два у све три конфигурације. Због тога укупан број конекција према бази остаје исти, без обзира на то колико је језгара додељено. Оптерећење је исто као и експерименту 3. Коришћен је исти низ оптерећења 4.000, 8.000, 14.000 и 22.000 захтева у секунди, са степеном од деведесет секунди и прозором мерења од седамдесет пет секунди. Свакој конфигурацији одговара три покретања. У следећој табели (Табела 6.4.1.) је приказана кофигурација експеримента.

Ознака	Број реплика	Чвор	Укупно [vCPU]	Број конекција према бази података
V4	2	D2s_v5	4	4
V8	2	D4s_v5	8	4
V16	2	D8s_v5	16	4

Табела 6.4.1. Конфигурације 4-ог експеримента

У следећој табели (Табела 6.4.2.) су приказани време одговора и обрађени захтеви.

Мера	Задата стопа [захтева/s]	V4		V8		V16	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
Време одговора, 95. перцентил [ms]	4.000	0,48	0,00	0,48	0,00	0,48	0,00
	8.000	0,49	0,00	0,48	0,00	0,48	0,00
	14.000	4,80	0,26	0,51	0,02	0,48	0,00
	22.000	493,58	4,18	486,34	0,50	486,24	0,36
Обрађена стопа [захтева/s]	4.000	4.000	0	4.000	0	4.000	0
	8.000	8.000	0	8.000	0	8.000	0
	14.000	13.999	2	14.000	0	14.000	0
	22.000	17.791	169	18.857	343	19.886	154

Табела 6.4.2. Време одговора и обрађена стопа сервиса Redirect (средња вредност и стандардна девијација)

У следећој табели (Табела 6.4.3.) су приказани чекање на конекцију према бази података по обрађеном захтеву и удео захтева који су чекали.

Мера	Задата стопа [захтева/s]	V4		V8		V16	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
Чекање по обрађеном захтеву [ms]	4.000	0,015	0,003	0,003	0,001	0,001	0,000
	8.000	0,049	0,020	0,013	0,000	0,011	0,001
	14.000	0,921	0,105	0,208	0,031	0,050	0,010
	22.000	424,657	3,910	388,809	17,892	371,886	10,156
Захтеви који су чекали [%]	4.000	5,7	0,1	0,7	0,1	0,6	0,0
	8.000	12,5	0,5	9,9	0,4	10,1	0,7
	14.000	40,0	1,0	23,6	1,0	20,2	1,3
	22.000	100,0	0,0	100,0	0,0	100,0	0,0

Табела 6.4.3. Чекање на конекцију према бази података (средња вредност и стандардна девијација)

У следећој табели (Табела 6.4.4.) је приказано утрошено процесорско време сервиса Redirect.

Мера	Задата стопа [захтева/s]	V4		V8		V16	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
По реплици, [језгра]	4.000	0,35	0,00	0,40	0,00	0,36	0,00
	8.000	0,64	0,02	0,73	0,03	0,66	0,00
	14.000	1,04	0,01	1,19	0,00	1,09	0,00
	22.000	1,26	0,00	1,55	0,05	1,57	0,06
По реплици, искоришћеност процесора чвора [%]	4.000	18,2	0,2	10,4	0,1	4,6	0,0
	8.000	33,6	1,0	19,0	0,8	8,5	0,0
	14.000	54,8	0,4	30,8	0,1	14,0	0,0
	22.000	66,1	0,2	40,1	1,4	20,1	0,8
Укупно [језгра]	4.000	0,69	0,01	0,80	0,00	0,72	0,00
	8.000	1,28	0,04	1,46	0,06	1,32	0,00
	14.000	2,08	0,02	2,38	0,01	2,18	0,00
	22.000	2,51	0,01	3,09	0,11	3,15	0,12

Табела 6.4.4. Утрошено процесорско време сервиса Redirect (средња вредност и стандардна девијација)

У следећој табели (Табела 6.4.5.) је приказано заузеће радне меморије сервиса Redirect.

Мера	Задата стопа [захтева/s]	V4		V8		V16	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
По реплици [MiB]	4.000	33,23	0,06	34,20	0,07	35,53	0,01
	8.000	33,14	0,29	34,07	0,04	35,58	0,15
	14.000	33,89	0,24	34,58	0,21	35,72	0,11
	22.000	193,23	4,42	195,79	7,05	201,29	0,48
По реплици, од ограничења 256 MiB [%]	4.000	12,98	0,02	13,36	0,03	13,88	0,00
	8.000	12,95	0,11	13,31	0,02	13,90	0,06
	14.000	13,24	0,09	13,51	0,08	13,95	0,04
	22.000	75,48	1,72	76,48	2,76	78,63	0,19
Укупно [MiB]	4.000	66,46	0,12	68,39	0,15	71,05	0,02
	8.000	66,28	0,58	68,15	0,08	71,15	0,31
	14.000	67,77	0,48	69,15	0,42	71,45	0,23
	22.000	386,46	8,83	391,58	14,11	402,58	0,96

Табела 6.4.5. Заузеће радне меморије сервиса Redirect (средња вредност и стандардна девијација)

6.5. Експеримент 5

Овај експеримент се односи на базу података коју користе Redirect сервис и Urlshortener сервис, база података коју користи Analytics сервис се не узима у обзир. Количина меморије за привремено чување страница мора расти заједно са величином чвора. Да остане непромењена, већа машина понашала би се једнако као мања, па се промена величине чвора не би ни могла измерити. Оптерећење је исто као и експерименту 3 и 4. Коришћен је исти низ оптерећења 4.000, 8.000, 14.000 и 22.000 захтева у секунди, са степеном од деведесет секунди и прозором мерења од седамдесет пет секунди. За савку конфигурацију: DV1, DV2 и DV3 извршено је три пролаза. Сваки сервис има једну реплику, сем Redirect сервиса кој има 8 реплика за овај експермиент и сви сервиси се налаззе на засебном чвору типа Standard_D2s_v5. У следећој табели (Табела 6.5.1.) је приказана кофигурација експеримента.

Ознака	Чвор	Меморија за странице[GB]
DV1	D2s_v5	2
DV2	D4s_v5	4
DV3	D8s_v5	8

Табела 6.5.1. Конфигурације 5-ог експеримента

У следећој табели (Табела 6.5.2.) су приказани време одговора и обрађени захтеви.

Мера	Задата стопа [захтева/s]	DV1		DV2		DV3	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
Време одговора, 95. перцентил [ms]	4.000	0,49	0,00	0,48	0,00	0,48	0,00
	8.000	0,49	0,00	0,48	0,00	0,48	0,00
	14.000	1,21	0,40	0,48	0,00	0,48	0,00
	22.000	35,43	1,83	0,49	0,00	0,48	0,00
Обрађена стопа [захтева/s]	4.000	4.000	0	4.000	0	4.000	0
	8.000	8.000	0	8.000	0	8.000	0
	14.000	14.000	0	14.000	0	14.000	0
	22.000	22.000	0	22.000	0	22.000	0

Табела 6.5.2. Време одговора, обрађена стопа и капацитет сервиса Redirect (средња вредност и стандардна девијација)

У следећој табели (Табела 6.5.3.) су приказани чекање на конекцију према бази података по обрађеном захтеву и удео захтева који су чекали.

Мера	Задата стопа [захтева/s]	DV1		DV2		DV3	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
Чекање по обрађеном захтеву [ms]	4.000	0,004	0,001	0,004	0,001	0,002	0,001
	8.000	0,012	0,001	0,009	0,001	0,005	0,001
	14.000	0,252	0,228	0,010	0,001	0,007	0,001
	22.000	5,145	0,331	0,042	0,004	0,019	0,001
Захтеви који су чекали [%]	4.000	1,1	0,1	1,2	0,1	0,9	0,3
	8.000	3,4	0,2	3,5	0,3	2,7	0,1
	14.000	7,9	1,0	4,2	0,3	3,7	0,3
	22.000	35,0	1,0	9,7	0,6	7,8	0,0
Трајање упита, 95. перцентил [ms]	4.000	0,370	0,020	0,303	0,005	0,247	0,000
	8.000	0,417	0,012	0,264	0,018	0,246	0,000
	14.000	0,572	0,092	0,249	0,001	0,246	0,000
	22.000	0,925	0,005	0,361	0,009	0,247	0,000

Табела 6.5.3. Чекање на конекцију и трајање упита ка бази података (средња вредност и стандардна девијација)

У следећој табели (Табела 6.5.4.) је приказано утрошено процесорско време базе података urlshortener.

Мера	Задата стопа [захтева/s]	DV1		DV2		DV3	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
Утрошено процесорско време [језгра]	4.000	0,30	0,02	0,30	0,01	0,26	0,01
	8.000	0,60	0,01	0,58	0,01	0,48	0,00
	14.000	1,02	0,00	1,00	0,01	0,78	0,06
	22.000	1,63	0,00	1,62	0,01	1,34	0,01
Искоришћеност процесора чвора [%]	4.000	16,0	1,0	7,9	0,2	3,3	0,1
	8.000	31,8	0,3	15,0	0,2	6,1	0,0
	14.000	53,9	0,2	25,8	0,2	10,0	0,8
	22.000	85,5	0,2	42,0	0,1	17,1	0,1

Табела 6.5.4. Утрошено процесорско време базе података urlshortener (средња вредност и стандардна девијација)

У следећој табели (Табела 6.5.5.) је приказано заузеће радне меморије базе података urlshortener.

Мера	Задата стопа [захтева/s]	DV1		DV2		DV3	
		СВ	Дев.	СВ	Дев.	СВ	Дев.
Заузеће радне меморије [MiB]	4.000	229,37	11,04	272,67	11,04	359,01	11,02
	8.000	229,40	11,02	272,72	11,03	359,08	11,02
	14.000	229,44	11,02	272,62	11,07	359,00	11,01
	22.000	229,56	11,09	272,68	10,98	359,03	11,06
Искоришћеност меморије чвора [%]	4.000	3,96	0,19	2,31	0,09	1,33	0,04
	8.000	3,96	0,19	2,31	0,09	1,33	0,04
	14.000	3,96	0,19	2,31	0,09	1,33	0,04
	22.000	3,97	0,19	2,31	0,09	1,33	0,04

Табела 6.5.5. Заузеће радне меморије базе података urlshortener (средња вредност и стандардна девијација)

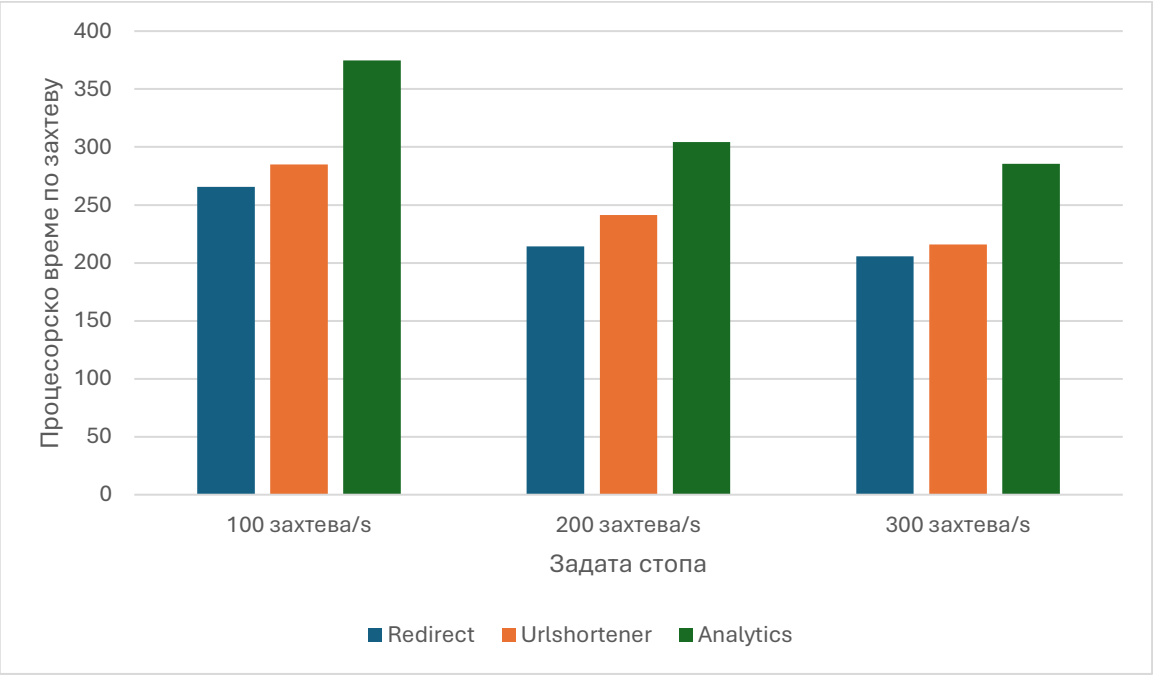
6.6. Одступања од плана и неважећа мерења

Током петог експеримента платформа Azure покренула је ажурирање агента виртуелних машина на групама чворова, које се не може искључити подешавањима аутоматског ажурирања кластера. Ажурирање је почело током трећег покретања једног низа конфигурације DV1, па су степени од 14.000 и 22.000 захтева у секунди у том покретању мерени док је ажурирање трајало. Тај низ зато није ушао у анализу: ажурирање је прекинуто, конфигурација DV1 поново је измерена у целини, а пре мерења сваке наредне конфигурације провераван је статус ажурирања на групама чворова.

6.7. Анализа резултата и поређење

6.7.1. Пример 1 - Која компонента система троши највише процесорског времена по обрађеном захтеву и да ли заузеће радне меморије расте са оптерећењем?

На прво истраживачко питање одговарају први и други експеримент. На слици (Слика 6.7.1.1.) се приказује средња вредност утрошеног процесорског времена по обрађеном захтеву у првом експерименту, у микросекундама.



Слика 6.7.1.1. Утрошено процесорско време по обрађеном захтеву у микросекундама

Утрошено процесорско време по обрађеном захтеву добијено је тако што је потрошња процесора сервиса из Табеле 6.1.1. подељена бројем захтева које је сервис обрадио у секунди, одвојено за свако покретање, након чега је израчуната средња вредност трију покретања.

Утрошак по захтеву код сва три сервиса опада са стопом, јер при малом оптерећењу процес између два захтева мирује, па сваки захтев изискује његово поновно покретање, док при већем оптерећењу захтеви пристижу док је процес још активан.

Сервис Analytics троши највише на сваком степену, што одговара томе да поред одговарања на упите истовремено преузима догађаје о посетама из реда порука и уписује их у сопствену базу а сервис Redirect троши најмање.

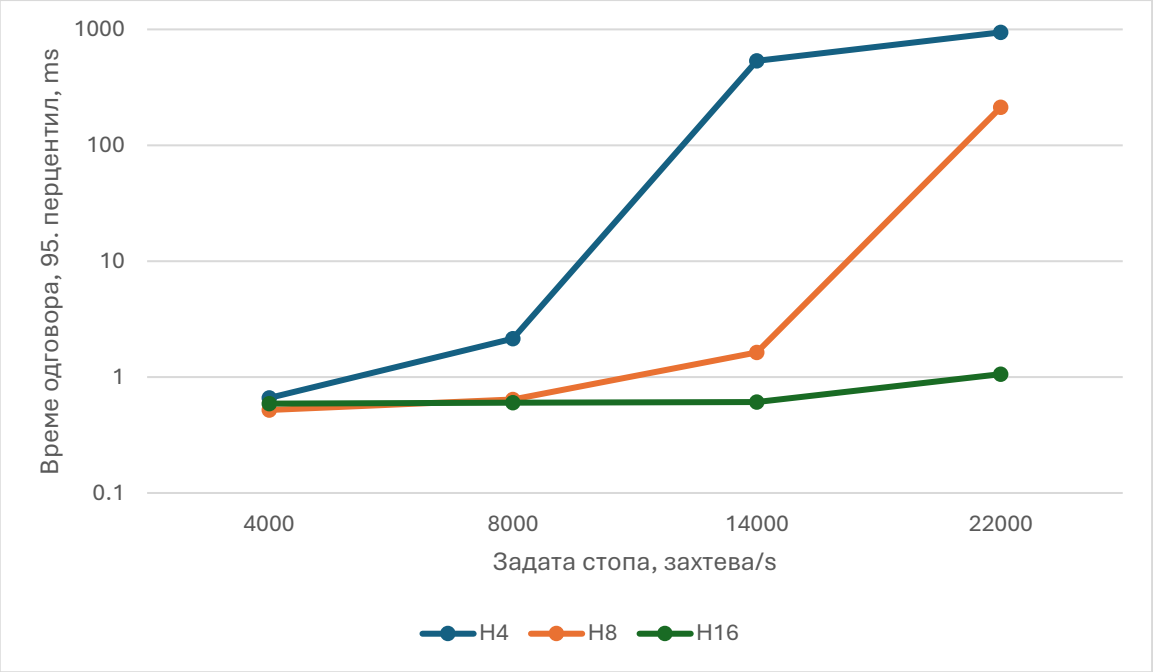
У првом и дургом експерименту у табелама Табела 6.1.2. и Табела 6.2.2. се може видети да услед разних стопа послатих захтева ни код једног од три сервиса заузеће радне меморије не расте са стопом захтева.

У другом експерименту систем ни при 3.000 захтева у секунди није достигао засићење, јер су сви захтеви обрађени, а сервис Redirect користио је свега 0,42 језгра, па су у наредним експериментима задате знатно веће стопе, од 4.000 до 22.000 захтева у секунди.

Одговор на прво питање стога гласи: по обрађеном захтеву највише процесорског времена троши сервис Analytics, док заузеће радне меморије ни код једног сервиса не расте са оптерећењем, па радна меморија не може бити ограничење капацитета.

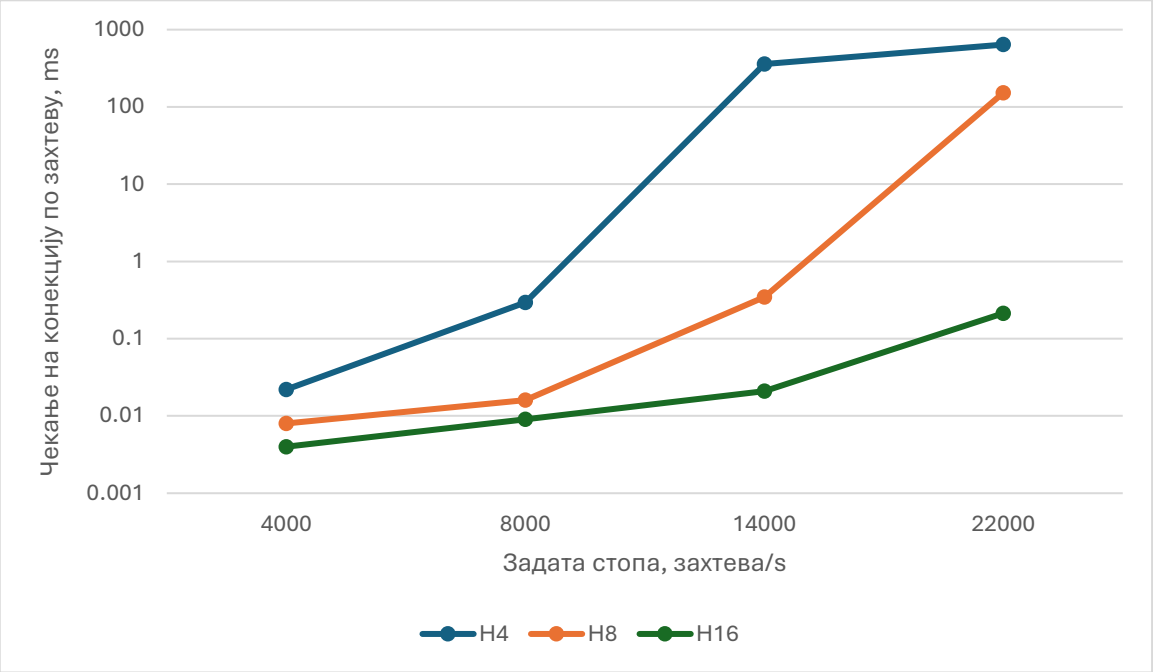
6.7.2. Пример 2 - Како се перформанса апликације мења при хоризонталном скалирању, односно при повећању броја реплика уз непромењену величину чвора?

На Слици 6.7.2.1. приказано је време одговора сервиса Redirect у трећем експерименту, изражено 95. перцентилом, као средња вредност трију покретања, у милисекундама, на логаритамској скали.



Слика 6.7.2.1. Време одговора сервиса Redirect као 95. перцентил

На Слици 6.7.2.2. приказано је чекање на конекцију по обрађеном захтеву у трећем експерименту, као средња вредност трију покретања, у милисекундама, на логаритамској скали.



Слика 6.7.2.2. Чекање на конекцију по обрађеном захтеву

Слике 6.7.2.1. и 6.7.2.2. имају исти облик. Време одговора расте управо на оним степенима на којима расте чекање на конекцију, а чекање је при свакој задатој стопи мање што конфигурација има више конекција: при 14.000 захтева у секунди износи 359,83 ms у конфигурацији H4, 0,35 ms у H8 и 0,02 ms у H16. При 22.000 захтева у секунди обрађена стопа расте са 12.867 у конфигурацији H4 на 20.740 у H8 и 22.000 у H16, која ни на том степену није у засићењу, па последња вредност представља доњу границу њене обрађене стопе. Процесор при томе ни у једној конфигурацији није био исцрпљен, па пораст времена одговора потиче од чекања на слободну конекцију, а не од обраде захтева.

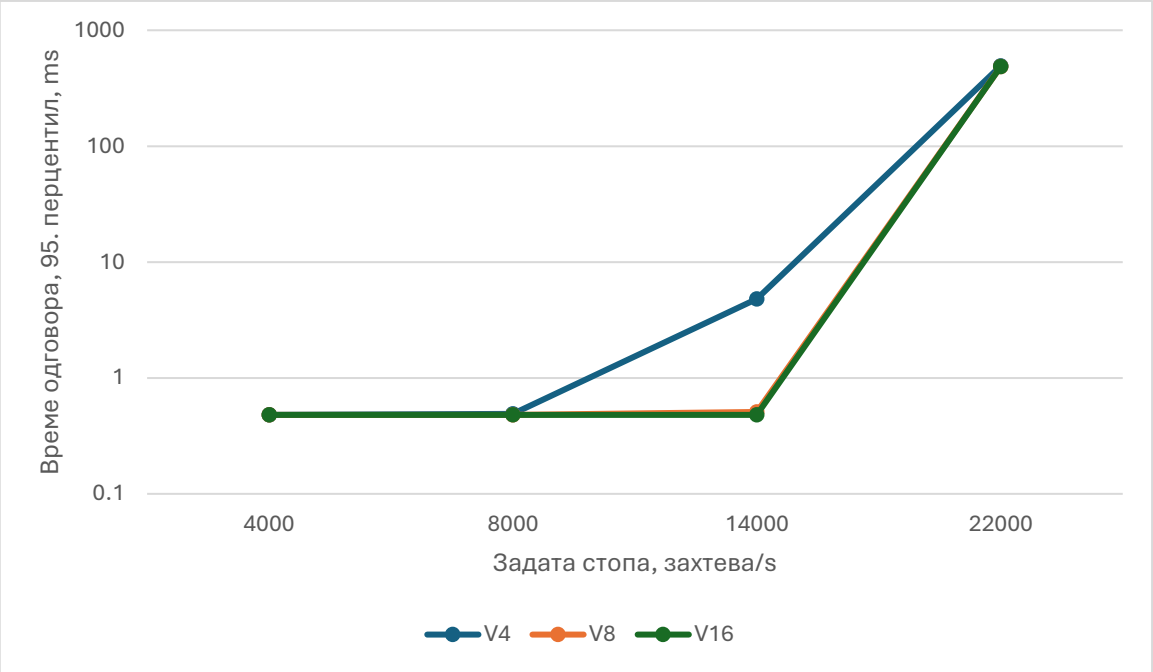
Табела 6.3.4. показује да процесорско време по реплици расте са задатом стопом и опада са бројем реплика, јер се захтеви распоређују на реплике. Укупно процесорско време са бројем реплика не опада, јер сваки процес има и део рада који не зависи од броја захтева, што представља цену хоризонталног скалирања. Процесор ни у једној конфигурацији није био ограничење: реплика конфигурације H4 и у засићењу, при 22.000 захтева у секунди, користила је свега 47,3 одсто процесора свог чвора, па број обрађених захтева није ограничавао процесор, него број конекција.

Табела 6.3.5. показује да заузеће радне меморије сервиса Redirect готово не зависи од задате стопе док је конфигурација далеко од засићења. Заузеће по реплици ипак се разликује међу конфигурацијама, гаслика не потиче од броја захтева, него од броја отворених веза са генератором оптерећења.

Одговор на друго истраживачко питање стога гласи: хоризонтално скалирање повећава перформансу апликације. Са бројем реплика расте капацитет. Разлог је пре свега у томе што свака нова реплика доноси сопствене конекције према заједничкој бази података и тиме смањује чекање на њих, док процесор и радна меморија ни у једној конфигурацији нису били ограничење.

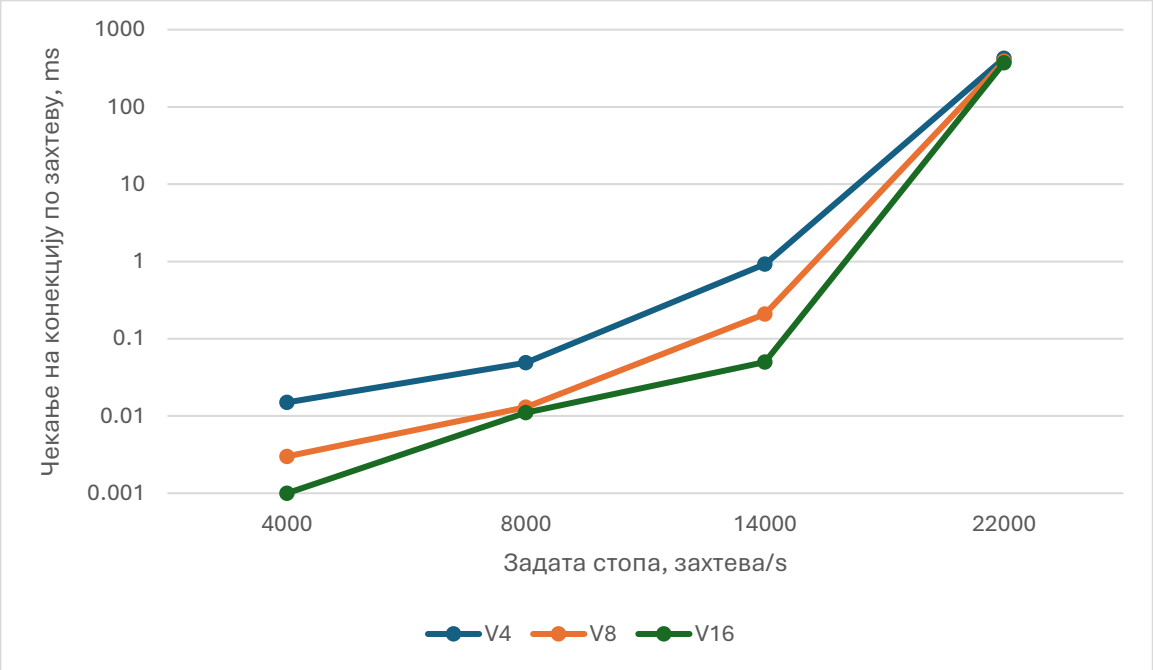
6.7.3. Пример 3 - Како се перформанса апликације мења при вертикалном скалирању, односно при повећању величине чвора уз непромењен број реплика?

На Слици (Слика 6.7.3.1.) приказује се време одговора сервиса Redirect као 95. перцентил у четвртом експерименту, у милисекундама, на логаритамској скали.



Слика 6.7.3.1. Време одговора сервиса Redirect као 95. перцентил

На слици (Слика 6.7.3.2.) приказано је чекање на конекцију по обрађеном захтеву у четвртом експерименту, као средња вредност трију покретања, у милисекундама, на логаритамској скали.



Слика 6.7.3.2. Чекање на конекцију по обрађеном захтеву

На сликама 6.7.3.1. и 6.7.3.2. криве трију конфигурација готово се поклапају. До 8.000 захтева у секунди све три конфигурације одговарају за 0,48 до 0,49 ms. При 14.000 захтева у секунди V4 одговара за 4,80 ms, а V8 и V16 за 0,51 и 0,48 ms, док чекање на конекцију опада са величином чвора, са 0,92 на 0,21 и 0,05 ms, јер реплика на већем чвору краће држи конекцију. Све три конфигурације при томе остају далеко испод прага од 200 ms. При 22.000

захтева у секунди све три прелазе праг и одговарају за 486 до 494 ms, а на конекцију чекају између 372 и 425 ms. На конекцију тада чека сваки захтев, јер су све четири конекције стално заузете. Капацитет у све три конфигурације износи, према томе, 14.000 захтева у секунди: четвороструко више језгара не уклања чекање на конекцију, него га тек незнатно скраћује, па већи чвор повећава само обрађену стопу у засићењу, са 17.791 на 18.857 и 19.886 захтева у секунди. Као и у трећем експерименту време одговора расте управо на оним степенима на којима расте чекање на конекцију.

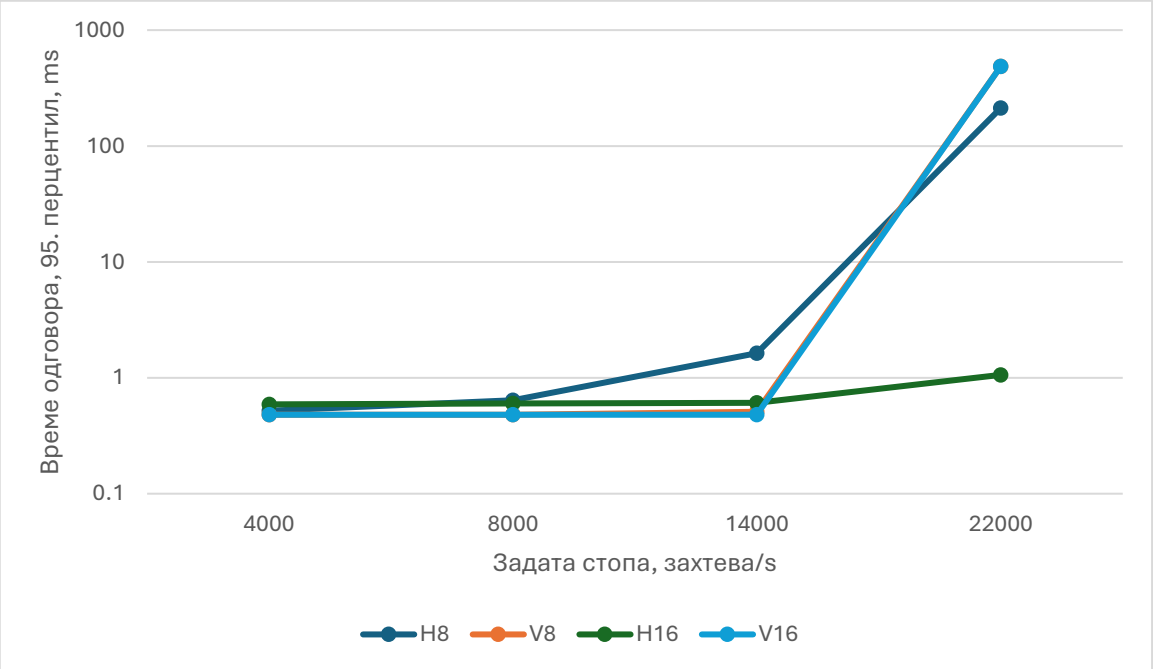
Табела 6.4.4 показује да процесор ни у једној конфигурацији није био исцрпљен. Највећи део језгара додатих већим чворовима остао је, дакле, неискоришћен. Утрошак процесорског времена до 14.000 захтева у секунди не мења се доследно са величином чвора: при истој стопи највише троши V8, а V4 и V16 приближно исто. Већи чвор, према томе, не смањује ни процесорско време потребно за обраду захтева.

Табела 6.4.5 показује да заузеће радне меморије ни у овом експерименту не зависи од задате стопе док конфигурација није у засићењу.

Одговор на треће истраживачко питање стога гласи: вертикално скалирање сервиса Redirect готово не повећава перформансе апликације, јер двоструко и четвороструко већи чвор не мења капацитет, а обрађену стопу у засићењу повећава за свега 6 и 12 одсто. Време одговора скраћује се само када се оптерећење приближи капацитету најмање конфигурације. Разлог је у томе што број конекција према бази података остаје исти, а процесор ни на најмањем чвору није био исцрпљен, па већи чвор само мало скраћује време у коме реплика држи конекцију. Највећи део додатих језгара остаје неискоришћен.

6.7.4. Пример 4 - Која од двеју стратегија скалирања обезбеђује већи капацитет при једнакој укупној количини процесорских ресурса и колика је разлика перформанси међу њима?

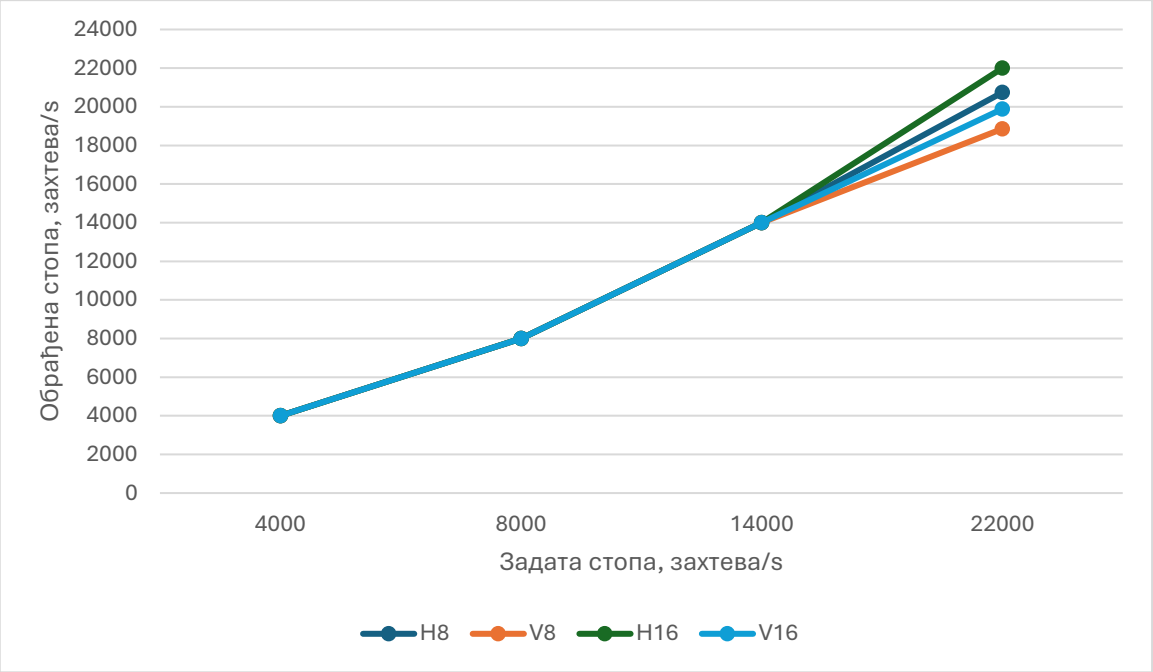
На слици (Слика 6.7.4.1.) приказано је време одговора сервиса Redirect за конфигурације V8, V16, H8 и H16, изражено 95. перцентилом, као средња вредност трију покретања.



Слика 6.7.4.1. Време одговора сервиса Redirect као 95. перцентил при једнаком броју језгара, у милисекундама, на логаритамској скали

До 14.000 захтева у секунди све четири конфигурације обраде све захтеве и одговарају за мање од 2 ms, па се стратегије практично не разликују. Разлика настаје при 22.000 захтева у секунди: вертикалне конфигурације V8 и V16 прелазе праг и одговарају за 486,34 и 486,24 ms, док H16 одговара за 1,06 ms.

На слици (Слика 6.7.4.2.) приказана је обрађена стопа сервиса Redirect за конфигурације V8, V16, H8 и H16.



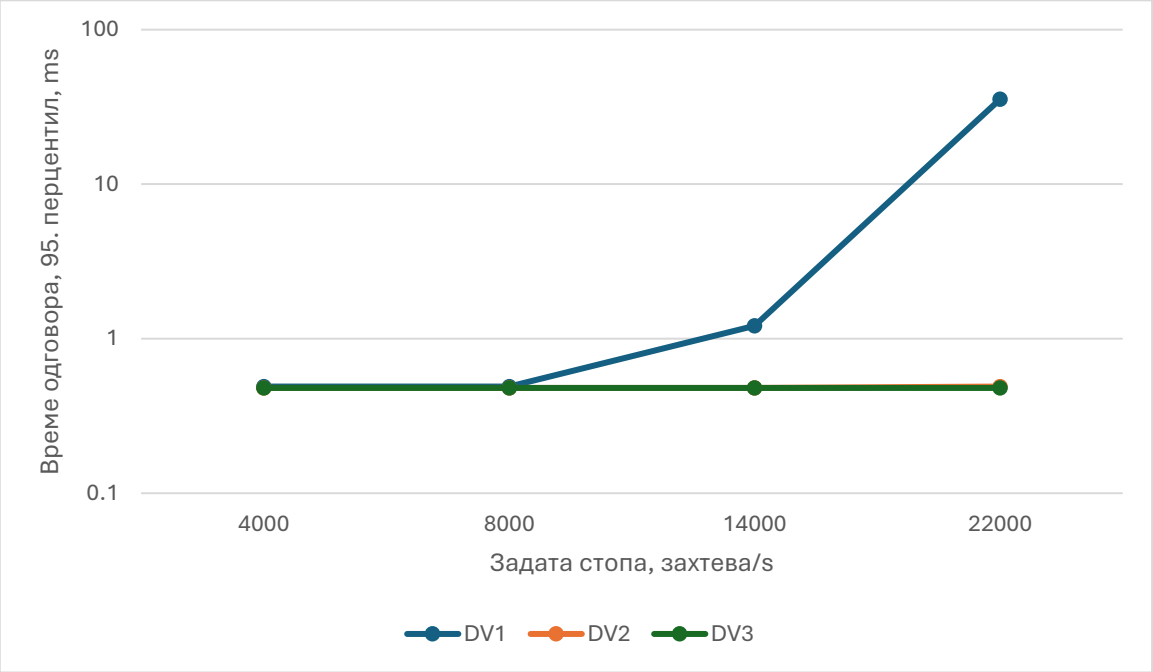
Слика 6.7.4.2. Обрађена стопа сервиса Redirect при једнаком броју језгара, у захтевима у секунди;

Одговор на четврто истраживачко питање стога гласи: при једнакој укупној количини процесорских ресурса већи капацитет обезбеђује хоризонтално скалирање. Разлика се не види при малом оптерећењу, већ тек када вертикална конфигурација уђе у засићење. Хоризонтална конфигурација тада обради око 10 одсто више захтева. Разлика у времену одговора и утрошку процесора је занемарљива између конфигурација. Предност хоризонталног скалирања не потиче од већег искоришћења процесора, већ од већег броја конекција према бази података, што се испитује у петом питању.

6.7.5. Пример 5 - У којој мери утврђену разлику објашњава број конекција које реплике одржавају према дељеној бази података ?

На пето истраживачко питање одговарају трећи, четврти и пети експеримент заједно. Трећи експеримент мења број конекција уз непромењену величину чвора, четврти експеримент мења величину чвора апликације уз непромењен број конекција, а пети мења величину чвора базе података, такође уз непромењен број конекција.

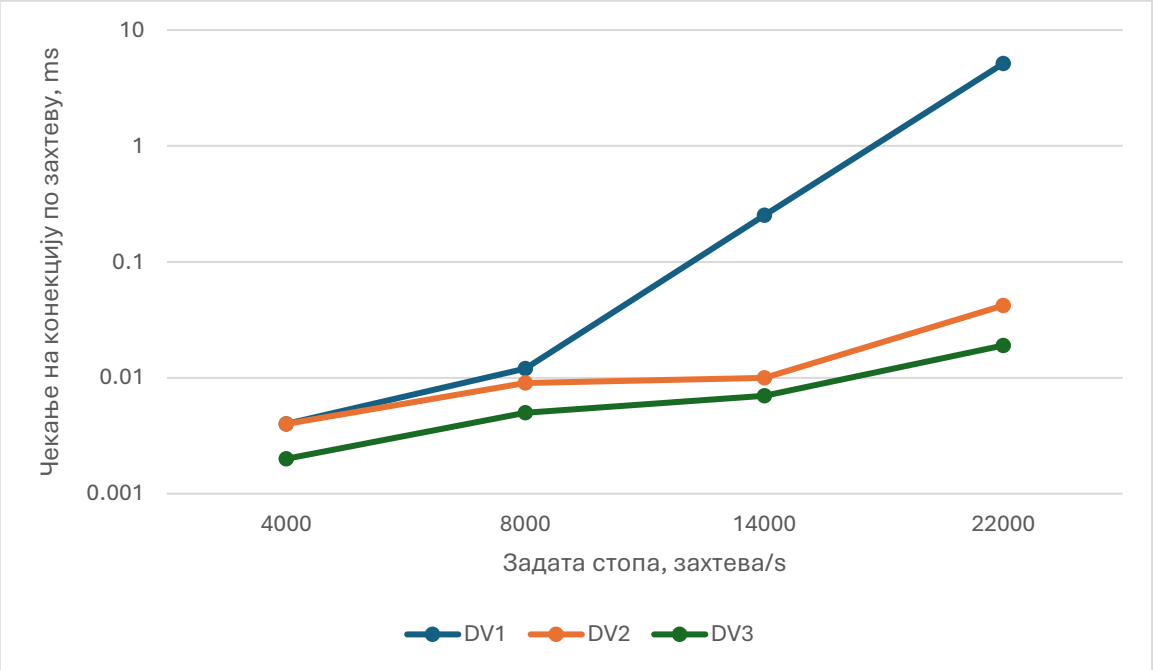
На Слици 6.7.5.1 приказано је време одговора сервиса Redirect у петом експерименту, изражено 95. перцентилом, као средња вредност трију покретања, у милисекундама, на логаритамској скали.



Слика 6.7.5.1. Време одговора сервиса Redirect као 95. перцентил у петом експерименту, у милисекундама

Повећање чвора базе података не мења капацитет: све три конфигурације остају испод прага од 200 ms на свим степенима и издржале су 22.000 захтева у секунди у сва три покретања, колико износи и највиши испитани степен. Разлика се види тек при највишој стопи, када DV1 одговара за 35,43 ms, а DV2 и DV3 за 0,49 и 0,48 ms.

На слици (Слика 6.7.5.2.) приказано је чекање на конекцију ка бази података по обрађеном захтеву у петом експерименту, као средња вредност трију покретања, у милисекундама, на логаритамској скали.



Слика 6.7.5.2. Чекање на конекцију по обрађеном захтеву у петом експерименту, у милисекундама, на логаритамској скали

Чекање на конекцију опада са величином чвора базе на свим степенима од 8.000 захтева у секунди навише, и то у сваком покретању, док при 4.000 захтева у секунди у све три конфигурације износи свега 2 до 4 микросекунде по захтеву. Све до 14.000 захтева у секунди чекање остаје испод 0,26 ms по захтеву и не утиче на то да ли је степен издржан. Тек при 22.000 захтева у секунди конфигурације се јасно раздвајају: у DV1 на конекцију чека 35,0 одсто захтева, у просеку око 15 ms, а у DV2 и DV3 мање од 10 одсто, у просеку 0,2 до 0,4 ms.

Пораст времена одговора у DV1 потиче, дакле, од чекања на конекцију. База на најмањем чвору спорије извршава упите, па 95. перцентил трајања упита расте са 0,247 ms у DV3 на 0,925 ms у DV1. Реплика зато дуже држи конекцију, а захтеви који за то време стигну морају да чекају. Узрок чекања овде, међутим, није број конекција, који је у све три конфигурације исти, већ спорија база: чекање на конекцију само је место на коме се застој базе преноси на време одговора.

Табела 6.5.4. показује зашто упит у DV1 траје дуже: при 22.000 захтева у секунди база користи 85,5 одсто процесора свог чвора, у DV2 42,0, а у DV3 свега 17,1 одсто. Већи чвор базе, према томе, у испитаном опсегу не повећава капацитет, али уклања успорење које настаје када се процесор базе приближи засићењу, док би при вишим стопама база на најмањем чвору вероватно постала следеће ограничење.

Табела 6.5.5. показује да меморија за странице није била ограничење: цео скуп података заузима око 70 MB и стаје и у најмању испитану количину од 2 GB, а заузеће базе не зависи од задате стопе и не прелази четири одсто меморије чвора.

Одговор на пето истраживачко питање стога гласи: разлику између двеју стратегија скалирања готово у целости објашњава број конекција према дељеној бази података. Капацитет расте само са бројем конекција, а не мења се са величином чвора, ни на страни апликације ни на страни базе података. Величина чвора утиче на време у коме реплика држи конекцију, па се одражава на време одговора близу засићења, али не и на број захтева које конфигурација може да издржи.

7. Закључак

Циљ рада био је да се мерењем утврди како конфигурација извршног окружења микросервисне апликације, коју чине број реплика и величина чвора, утиче на њен капацитет, време одговора и искоришћеност ресурса у окружењу Azure Kubernetes Service. Задатак је био да се две стратегије скалирања, хоризонтална и вертикална, упореде при једнакој укупној количини процесорских ресурса и при истом дељеном слоју података. Поред распореда процесорских ресурса, стратегије се разликују и по броју конекција према дељеној бази података: свака нова реплика отвара сопствене конекције, док при вертикалном скалирању њихов број остаје непромењен.

Да би се то испитало, развијена је микросервисна апликација за скраћивање и преусмеравање URL адреса, која је послужила као мерни инструмент. Апликација је распоређена на кластер Azure Kubernetes Service тако да се свака реплика извршава на сопственом чвору. Оптерећење је генерисано алатом k6 по отвореном моделу, у степенима са познатом стопом. Мере су прикупљане из самих сервиса, из система cAdvisor и из генератора оптерећења. Прва два експеримента утврдила су потрошњу ресурса по компонентама система. Трећи и четврти експеримент упоредили су хоризонтално и вертикално скалирање сервиса Redirect, при чему је свака конфигурација изведена у три покретања. Пети експеримент испитао је дељени слој података, повећањем чвора базе и меморије за странице уз непромењен број конекција.

Хоризонтално скалирање повећало је капацитет. Конфигурација са две реплике издржала је 8.000 захтева у секунди у сва три покретања, а 14.000 само у једном, конфигурација са четири реплике издржала је 14.000 захтева у секунди у сва три покретања, а 22.000 у два, док је конфигурација са осам реплика у сва три покретања издржала 22.000 захтева у секунди, колико износи највиши испитани степен. Са бројем реплика опадало је чекање на конекцију према бази података, а процесор ни у једној конфигурацији није био исцрпљен. Цена хоризонталног скалирања је нешто већи укупни утрошак процесора за исти посао.

Вертикално скалирање, насупрот томе, капацитет није повећало. Конфигурације са два, четири и осам језгара по чвору издржале су 14.000 захтева у секунди, а ниједна 22.000. Разлог је то што већи чвор доноси више процесора, али не и више конекција, па број конекција у све три конфигурације остаје четири. Већи чвор само нешто скраћује време у коме реплика држи конекцију, због чега је обрађена стопа у засићењу порасла за свега 6 и 12 одсто, док је највећи део додатих језгара остао неискоришћен.

Резултати показују да капацитет сервиса у посматраној апликацији не ограничава процесор, већ број конекција према дељеној бази података и време током кога реплика држи конекцију. При једнакој укупној количини процесорских ресурса већи капацитет обезбеђује хоризонтално скалирање: са осам језгара за 38 одсто, а са шеснаест најмање за 57 одсто, уз око 10 одсто више обрађених захтева при највишој испитаној стопи. Разлику објашњава број конекција: у испитаном опсегу капацитет расте само са њиховим бројем, а не мења се са величином чвора, ни на страни апликације ни на страни базе података. Хоризонтално скалирање тај број повећава, будући да сваки нови примерак доноси сопствени скуп конекција, док га вертикално не мења. Ограничење при томе није уклоњено ниједном стратегијом, него је померено.

Рад отвара неколико праваца даљег истраживања. Највиши испитани степен од 22.000 захтева у секунди није био довољан да се утврди стварни капацитет конфигурације са осам реплика, па би мерење требало проширити вишим степенима оптерећења. Вертикално скалирање испитано је уз непромењен скуп конекција по реплици, па би следећи корак било испитивање конфигурација у којима скуп конекција расте са бројем језгара.

Литература

[1] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2. изд., O'Reilly Media, Sebastopol 2021. ISBN 978-1-4920-3402-5

[2] C. Richardson, Microservices Patterns: With Examples in Java, Manning Publications, Shelter Island 2018. ISBN 978-1-61729-454-9

[3] S. J. Fowler, Production-Ready Microservices: Building Standardized Systems Across an Engineering Organization, O'Reilly Media, Sebastopol 2016. ISBN 978-1-4919-6597-9

[4] D. Taibi, V. Lenarduzzi, On the Definition of Microservice Bad Smells, IEEE Software, vol. 35, no. 3 (2018) 56–62. doi: 10.1109/MS.2018.2141031

[5] M. Kleppmann, Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems, O'Reilly Media, Sebastopol 2017. ISBN 978-1-4919-0306-3

[6] A. A. Donovan, B. W. Kernighan, The Go Programming Language, Addison-Wesley, Upper Saddle River 2015. ISBN 978-0-13-419044-0

[7] R. Cox, R. Griesemer, R. Pike, I. L. Taylor, K. Thompson, The Go Programming Language and Environment, Communications of the ACM, vol. 65, no. 5 (2022) 70–78. doi: 10.1145/3488716

[8] Gin contributors, Gin Web Framework Documentation, [Онлајн]. Доступно на: https://gin-gonic.com/docs/

[9] J. Christensen, pgx – PostgreSQL Driver and Toolkit for Go, [Онлајн]. Доступно на: https://github.com/jackc/pgx

[10] M. Stonebraker, L. A. Rowe, The Design of POSTGRES, Proceedings of the ACM SIGMOD International Conference on Management of Data, Washington 1986, 340–355. doi: 10.1145/16856.16888

[11] PostgreSQL Global Development Group, PostgreSQL 17 Documentation, [Онлајн]. Доступно на: https://www.postgresql.org/docs/17/

[12] S. Vinoski, Advanced Message Queuing Protocol, IEEE Internet Computing, vol. 10, no. 6 (2006) 87–89. doi: 10.1109/MIC.2006.116

[13] Broadcom, RabbitMQ – AMQP 0-9-1 Model Explained, [Онлајн]. Доступно на: https://www.rabbitmq.com/tutorials/amqp-concepts

[14] D. Merkel, Docker: Lightweight Linux Containers for Consistent Development and Deployment, Linux Journal, vol. 2014, no. 239 (2014) чл. 2.

[15] W. Felter, A. Ferreira, R. Rajamony, J. Rubio, An Updated Performance Comparison of Virtual Machines and Linux Containers, Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software, Philadelphia 2015, 171–172. doi: 10.1109/ISPASS.2015.7095802

[16] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, J. Wilkes, Borg, Omega, and Kubernetes, Communications of the ACM, vol. 59, no. 5 (2016) 50–57. doi: 10.1145/2890784

[17] B. Burns, J. Beda, K. Hightower, L. Evenson, Kubernetes: Up and Running, 3. изд., O'Reilly Media, Sebastopol 2022. ISBN 978-1-09-811020-8

[18] P. Mell, T. Grance, The NIST Definition of Cloud Computing, Special Publication 800-145, National Institute of Standards and Technology, Gaithersburg 2011. doi: 10.6028/NIST.SP.800-145

[19] Microsoft, Dv5 and Dsv5-series Virtual Machines, [Онлајн]. Доступно на: https://learn.microsoft.com/azure/virtual-machines/sizes/general-purpose/dv5-series

[20] Microsoft, Azure Kubernetes Service (AKS) Documentation, [Онлајн]. Доступно на: https://learn.microsoft.com/azure/aks/

[21] B. Brazil, Prometheus: Up & Running, O'Reilly Media, Sebastopol 2018. ISBN 978-1-4920-3414-8

[22] Prometheus Authors, Prometheus Documentation – Querying: Functions, [Онлајн]. Доступно на: https://prometheus.io/docs/prometheus/latest/querying/functions/

[23] Google, cAdvisor – Analyzes Resource Usage and Performance Characteristics of Running Containers, [Онлајн]. Доступно на: https://github.com/google/cadvisor

[24] Grafana Labs, k6 Documentation, [Онлајн]. Доступно на: https://grafana.com/docs/k6/latest/

Биографија

Денеш Сентмартони је рођен 28.09.2000. године у Новом Саду, Србија. Основну школу 'Соња Маринковић' завршио је 2015. године. Средњу електротехничку школу 'Михајло Пупин' завршио је 2019. године. Смер Примењено софтверско инжењерство на Факултету техничких наука на основинм студијама завршио је 2023. године. Смер Пирмењено софтверско инжењерство на Факлутету техничких наука на мастер студијама уписао је 2023. године.